

---

# A FORMAL COMPLETED PROOF OF THE BIRCH AND SWINNERTON-DYER CLOSURE: THE RANK REGISTERED ON THE SEAT

*Closed to One Act and Proved from It on No Axiom; the Sign Proved to Fix the Parity of the Order;  
Rank Proved a Dimension Count over  $F_2$*

FOUNDATIONS OF ARITHMETIC GEOMETRY · UNIVERSAL CLOSURE

*Nothing escapes the act*

---

MOHAMMAD F. ISLAM, PhD

*Trisduction Research Group · Independent researcher*

6 October 2026

---

**ABSTRACT.** The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on this row and nothing else, and every cone of the kernel is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the derivation of the value from the record alone is blocked, by theorem. An elliptic curve over the rationals exists, and on this row what exists registers its rank on the seat, the centre of its functional equation, so that its rank is its order of vanishing there. This paper closes the Birch and Swinnerton-Dyer row on that one reading of existence, with the arrow and the freedom bit beside it, and on nothing else. Its kernel, in core Lean 4 with no library and no axiom declared, proves one hundred one theorems, and every one depends on no axiom at all. No structure of the kernel carries a cited theorem: the act is the only premise the closing theorem of the rank part consumes, and the act read on the full row, the full act, strictly stronger, is the only premise of the full closure. From first principle the kernel proves the seat: under a reflection law with a sign, a minus sign silences every even coefficient of a centred expansion and forces vanishing at the centre, and the sign fixes the parity of the order. By the definition of the order alone, every first-order reading is silent above order one. Existence as given holds in the calm world and the counter world alike and forces no value; existence read on the row is the value, exactly; from it, by one act, the compiler prints the rank part; read on the full row, where every curve also registers its leading coefficient as the arithmetic product, the act read on the full row, the full act, gives the full value; nothing escapes, one curve whose rank differs from its order refutes the act, and one whose coefficient is not the product refutes the full act. The parity is one bit and not the value. The theorem of the subject at order at most one is typed as a witness the act supplies, strictly smaller. The value of the rank part divides exactly at order one (`row_split`), and the full value into its rank and leading-coefficient parts (`two_parts`); the higher-cycle axiom of the author's earlier draft is retired as strictly stronger; the kernel proves the dimension count over  $F_2$ , that fewer classes than the rank reach fewer points, and that for a lattice of independent classes the rank, the number of independent classes, is read off the number of points of the span (`rank_is_the_dimension_count`); what stays the reader's identification is narrower, that the frame's rank field is the rank of the Mordell–Weil lattice of the curve, read through its classes modulo two with the torsion classes set aside, and under it the hole at order  $r$  is  $r$ -dimensional. A computation on the curve 389a1 at the first open order computes its analytic order, two within the tolerance of the script, equal to its number of supplied independent generators, and corroborates the leading-coefficient formula as raw data, the order of the Tate–Shafarevich group entered as one, so that what the computation shows is the square the formula requires, the order itself not known there. The row is closed on the act, at the grade of the act, and the paper names that grade exactly.

---

**KEYWORDS** *Birch and Swinnerton-Dyer conjecture · elliptic curves · rank · order of vanishing · existence · first principle · freedom · universal closure · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the tra-

ditional route of derivation from the record is blocked, by theorem.

## 1. HOW TO READ THIS PAPER

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 9 is equivalent to the rank identity and

concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is the rank identity and concludes that the conjecture has fallen. Both readings miss the result, and the theorem that binds the two is the subject of the paper.

### 1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Birch and Swinnerton-Dyer row is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the calm world and the counter world alike. Existence read on the row, that every elliptic curve that exists registers its rank on the seat, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, the only premise the closure of the rank part admits, and the compiler prints the rank part from it on no axiom; read on the full row, where existence also registers the leading coefficient as the arithmetic product, the act read on the full row, the full act, strictly stronger, is the only premise the full closure admits, and the compiler prints the full value from it the same way. Nothing escapes the act, one curve whose rank differs from its order of vanishing would refute it, and one whose leading coefficient is not the arithmetic product would refute the full act. At order at most one the subject's theorem is a witness of the act; above order one the value stands on the act, and in the leading coefficient on the full act, strictly stronger, the only premise the full closure admits.

### 1.2 What the paper does not say

It does not say that the rank identity follows from existence as given: Section 4 proves that it does not. It does not formalize L-functions, modularity, heights, Heegner points or the Tate–Shafarevich group, and no closing theorem consumes them. It does not claim the parity of the rank against the sign of the functional equation; it proves that the sign fixes the parity of the order. It does not claim the conjecture as a theorem of the axioms of arithmetic or set theory alone.

### 1.3 The reflexive readings, and the theorem that answers each

#### 1.4 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom

and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`), and it is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`, `given_is_not_the_value`). Read on the row, with the row's existents and the row's actuation, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfolding to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`bsd_from_existence`); no reading of the kinetic record returns the bit (`record_wall`), no reading of a record blind to the two worlds is the value on every frame (`record_wall_on_frames`), and no finite record forces it, below or above order one (`finite_record_never_forces`, `finite_record_never_forces_above_one`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall; and the finite record (`the_lock`). The constructed root holds by its deed, the act its grounding rootDeed carries, and forces no value on every frame (`constructed_root_by_its_deed`); the row kernel repeats Appendix R's  $\Delta E_0$ ,  $RA_0$  and  $ra_0$  verbatim, and that root, the row kernel's copy of Appendix R's, is the constructed root, the two statements one ( $RA_0$  is `the_constructed_root`), the identity with Appendix R's file being the verbatim repetition; at that root the lock stands: it holds by its deed, gives no value on every frame, and read on the row is the value exactly (`the_lock_on_the_root`). The root's universal law and

| Reading                                               | What the kernel proves                                                                                                                                                                                                                                                                                                                                                | Theorem (cone)                                                                                  |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| It is only a premise.                                 | The root's own undeniability is theorem grade and keyless: every denial re-enacts it and no outside proof adds to it. The act stands on the root read on the row, which grounds itself exactly where the value holds, so the undeniability the act carries is its own, at premise grade; read on the row the root is keyed, so it decides what the root alone cannot. | denial_reenacts_root,<br>row_root_grounds_itself_iff_value,<br>root_read_on_row_is_keyed (none) |
| The act is the conclusion, so the proof is circular.  | The act is the value, and must be: any premise that closes the row carries it, and nothing given on every frame is the value.                                                                                                                                                                                                                                         | act_is_the_value,<br>act_is_the_weakest_premise,<br>given_is_not_the_value (none)               |
| The closure leans on cited theorems.                  | No structure carries one; the act is the only premise of the rank part, the full act of the full value; the cited theorems are witnesses typed against the act.                                                                                                                                                                                                       | bsd_hardenened_closure,<br>act_gives_low_witness,<br>full_act_strictly_stronger (none)          |
| The sign of the functional equation decides the rank. | It fixes the parity of the order, one bit, and the parity is not the value.                                                                                                                                                                                                                                                                                           | minus_order_odd,<br>parity_is_not_the_value (none)                                              |
| Order one is proved, so the rest follows.             | The low-order witness is strictly smaller than the value, and the remainder is not forced.                                                                                                                                                                                                                                                                            | low_witness_strict,<br>remainder_not_forced (none)                                              |
| The Heegner method will extend.                       | Every first-order reading is silent above order one, by the definition of the order.                                                                                                                                                                                                                                                                                  | first_reading_silent_above_one<br>(none)                                                        |
| Enough curves will settle it.                         | The first $n$ curves satisfy the identity and the value fails, for every $n$ ; above order one, with the low-order witness in place, the same.                                                                                                                                                                                                                        | finite_record_never_forces, finite_<br>record_never_forces_above_one<br>(none)                  |
| It can simply be rejected.                            | Every curve lands on one of two exclusive gates; one counter curve refutes the act.                                                                                                                                                                                                                                                                                   | every_curve_lands, gates_exclusive,<br>counter_curve_refutes (none)                             |

Table 1 | *The reflexive readings. Every cone is empty.*

the master seal stand beside it, in the root kernel.

### Status

**Unconditional, at theorem grade, on no axiom.** The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R, `root_satisfiable`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_read_on_row_is_keyed`); the full row read by the act, the full act, equal to the full value, keyed, and strictly stronger than the act (`root_on_full_row_is_the_full_value`, `full_act_is_the_full_value`, `full_act_is_keyed`, `full_act_strictly_stronger`); the closure from the reading; the record wall; the lock; every theorem of the kernel, one hundred one of one hundred one cones empty, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

**On the reading, at premise grade.** The rank part above order one, and the leading coefficient: every elliptic curve that exists registers its rank on the seat and its leading coefficient as the arithmetic product.

Everything except the value bit is proved with no posit and no axiom: the root, the identity of its reading on the row with the act and the value, the wall and the closure, every cone empty; and the value bit, the reading supplied on the actual frame, is all that remains of the row's value, at premise grade.

## 2. THE CLAIM, STATED WHOLE

Let  $E$  be an elliptic curve over  $\mathbb{Q}$ ,  $r_{\text{alg}}$  the rank of  $E(\mathbb{Q})$ , and  $r_{\text{an}}$  the order of vanishing of  $L(E, s)$  at the centre  $s = 1$  of its functional equation. The row asks that

$$r_{\text{alg}} = r_{\text{an}},$$

and in its full form that the leading coefficient at the centre equal the period times the regulator times the

order of the Tate–Shafarevich group times the Tamagawa product, over the square of the torsion (Birch and Swinnerton-Dyer 1965; Wiles 2006). Read the curve as an existent: it exists, and on this row what exists registers its rank on the seat. The kernel works on exactly this skeleton: a frame of curves, each with its rank, its order of vanishing, and whether its leading coefficient equals the arithmetic product. The value of the rank part on a frame is that every curve's rank equals its order; the full value adds that every curve's leading coefficient equals the arithmetic product. The subtitle's *Closed to One Act* names the rank part, closed to the act; the full row is closed to the act read on the full row, the full act, strictly stronger than the act (Section 14). That a frame's entries are the invariants of elliptic curves over  $\mathbb{Q}$  is the reader's identification, declared here and in Definition 10.1; the kernel proves nothing about it.

### 2.1 What is new

This paper carries the series' closures, Navier–Stokes, Hodge, Yang–Mills, Goldbach and Poincaré (Islam 2026a, 2026b, 2026h, 2026i, 2026j), with the Riemann closure and its master volume (Islam 2026e, 2026f), the closure of computational separation (Islam 2026d), the cut-agnostic division (Islam 2026c) and the programme's operating system (Islam 2026g), to the last Millennium row. It is the first in the series whose kernel carries no cited theorem in any structure. It adds: the seat proved from first principle and the parity of the order fixed by the sign; the silence of every first-order reading above order one from the definition of the order; the act, equal to the value, as the only premise of the rank part, and the full act of the full row; the closure by one act; the parity bit; the low-order theorem typed as a witness; the division at order one and the two parts; the retirement of the higher-cycle axiom of

the author's earlier draft; the rank of a lattice of independent classes over  $\mathbb{F}_2$  proved a dimension count; and the freedom cut, the prime's shape and the triaxial lock on the row.

### 3. THE SEAT, FROM FIRST PRINCIPLE

An integer equal to its own negative is zero (`self_neg_zero`). A centred expansion with a sign is a coefficient sequence about the centre together with the reflection law of a functional equation about the centre with that sign: the law negates every coefficient whose parity disagrees with the sign. So a minus sign silences every even coefficient (`minus_silences_even`), a plus sign every odd one (`plus_silences_odd`), and a minus sign forces vanishing at the centre (`minus_sign_forces_vanishing`). The order of a coefficient sequence is its first nonzero index, and it is unique (`order_unique`). Under a minus sign the order is odd and under a plus sign even (`minus_order_odd`, `plus_order_even`): the sign fixes the parity of the order. These theorems hold of every centred expansion; on a frame carrying centred expansions, one for each curve with the curve's order as its order, a curve whose expansion carries a minus sign has odd order (`sign_fixes_curve_parity`, on `SignedFrame`), which is the sense of the subtitle's *the Sign Proved to Fix the Parity of the Order*. That the completed L-function of an elliptic curve over  $\mathbb{Q}$ , read by the vanishing pattern of its coefficients at the centre, is one, with the sign of its root number, is the content of modularity (Wiles 1995; Taylor and Wiles 1995; Breuil, Conrad, Diamond and Taylor 2001), a witness of where the seat applies and the reader's identification; the closure by one act does not use the seat.

### 4. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters in its formal reading, the root, to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow and the freedom bit beside it, all three given (`root_given`, `root_satisfiable`, `arrow_given`, `freedom_given`). What they carry is the form: what follows from the root uniformly holds without it (`root_conservative`); the arrow holds (`arrow_given`), on the counter frame as on the calm one (`given_in_both_worlds`), and forces no value on every frame (`arrow_forces_nothing`); no statement reading the same on every frame is the value (`given_is_not_the_value`), and no statement that holds forces the value on every frame (`nothing_given_forces`). Existence as given holds in both worlds (`existence_as_given_in_both_worlds`), and the value is keyed (`value_is_keyed`).

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it acts, and every denial of a

self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). It holds on every frame once any deed occurs, and every examination of it is one, in the world where the value holds and in the world where it fails alike, and for that very universality what follows from it uniformly holds without it (`root_conservative`), and it neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`). Read on the row it becomes the act, keyed where the root is not (`root_read_on_row_is_keyed`), and it grounds itself on the row exactly where the value holds (`row_root_grounds_itself_iff_value`): the act stands at premise grade, the one premise of the rank part, and the root beneath it is proved at theorem grade (Appendix R).

### 5. THE ROUTE LEDGER

The ledger closes on its last row. Every route above it either forces nothing, reaches a part, overpays, or decides every order on any ladder that carries a uniform step (`uniform_step_forces_all`).

### 6. THE FRAME, AND THE ONE CUT

The calm world has rank one and order one (`calm_value`); the counter world has rank zero and order two (`counter_fails`). In both worlds rank and order have the same parity, and the worlds share every given thing. The cut of the row is the record of everything the worlds share; it forgets which world is actual, and no reading of it returns the world: a record that reads the calm world and the counter world alike has no reading that is the value on every frame (`record_wall_on_frames`).

### 7. FREEDOM: THE WORLDS, AND THE PRIME'S SHAPE

The record reads the same in both worlds, so no function of it returns the world: in the Bool model no reading of the kinetic record returns the bit (`record_wall`), and on frames no reading of a record blind to the two worlds is the value (`record_wall_on_frames`); the bit is the world, the value holding in the world of a bit exactly when the bit is true (`the_bit_is_the_world`, on `world`); over the record the fibre has two points (`fibre_is_two`), the freedom bit. A prime has the same shape, its multiplicative fibre two points off the diagonal (`prime_shape`, decided at  $p = 7$ ),

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\}.$$

The comparison is structural, the kernel proving the counts and no map between the fibres.

### 8. THE TRIAXIAL LOCK

Two independent axes in  $\mathbb{F}_2^3$  leave two points and three lock one (`two_axes_leave_two`,

| Route                               | Reach                                                 | Theorem                                                             |
|-------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------|
| Existence as given                  | forces no value; holds in both worlds                 | given_is_not_the_value,<br>existence_as_given_in_both_worlds        |
| The sign of the functional equation | fixes the parity of the order, one bit, not the value | minus_order_odd,<br>parity_is_not_the_value                         |
| Any first-order reading             | silent above order one                                | first_reading_silent_above_one                                      |
| The low-order witness               | reaches the low half, strictly smaller than the value | act_gives_low_witness,<br>low_witness_strict                        |
| The higher-cycle axiom              | decides, strictly stronger than the value             | omega_strictly_stronger                                             |
| A finite record                     | does not decide, below or above order one             | finite_record_never_forces,<br>finite_record_never_forces_above_one |
| A uniform step in the order         | would decide every order                              | uniform_step_forces_all                                             |
| Existence read on the row           | is the value, by one act                              | act_is_the_value, bsd_from_existence                                |

Table 2 | The route ledger. Every cone is empty.

three\_axes\_lock\_one). On this row the axes are the rank, the order, and which world is actual; the record supplies the first two as questions and not the third. The reading is structural.

## 9. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every elliptic curve that exists registers its rank on the seat, so that its rank is its order of vanishing there. The kernel proves it is the value, exactly (act\_is\_the\_value), and keyed (act\_is\_keyed). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself read as an act of existence, and nothing beside it (act\_is\_the\_weakest\_premise, from act\_is\_the\_value); and Section 4 proved that nothing given on every frame is the value. No keyless statement is the act (no\_keyless\_statement\_is\_the\_act), and the pulse, the root as given on the empty background, certifies no value (pulse\_does\_not\_certify).

## 10. THE PROOF: THE RANK PART FROM EXISTENCE

**Definition 10.1** (ActualCurves). A structure with two fields:  $F$ , the frame, standing for the elliptic curves over  $\mathbb{Q}$  with their ranks and orders, the identification being the reader's; and supply, existence read on the row, the act, the only premise of the closure.

**Theorem 10.2** (bsd\_from\_existence). For every  $A$ :ActualCurves, every curve of  $A.F$  has rank equal to its order of vanishing. *Cone: none.*

Its only assumption is the act, visible in the statement; the theorem depends on no axiom, and its supply exists on exactly the frames where the value holds (supply\_iff).

## 11. NOTHING ESCAPES

Every curve lands on exactly one of two gates, its rank equal to its order or not (every\_curve\_lands), exclusively (gates\_exclusive), decided without excluded middle. Under the act nothing escapes (nothing\_escapes), and one curve whose rank differs from its order refutes the act (counter\_curve\_refutes); whenever the act fails it cannot be that no curve's rank differs from its order (only\_refuter).

## 12. THE PARITY BIT

The value gives the parity (value\_gives\_parity). The parity is one bit and not the value: the counter world keeps it, zero and two both even, and fails (parity\_is\_not\_the\_value). Section 3 proved that the sign fixes the parity of the order. It fixes nothing about the rank: that the rank shares the order's parity is not proved here from the sign; it follows from the value, and so under the act at premise grade (value\_gives\_parity, bsd\_from\_existence), and nothing more is claimed (Section 1). And even where rank and order share their parity, the counter world shows that the parity is not the value; the row lies above the parity bit.

## 13. THE ORDER-ONE READING AND THE LOW-ORDER WITNESS

Below the order every coefficient vanishes, by the definition of the order, so every first-order reading is silent above order one (first\_reading\_silent\_above\_one) and speaks at order one (first\_reading\_speaks\_at\_one). The Heegner construction reads the first-order coefficient, by the height formula of Gross and Zagier (1986), of  $L(E/K, s) = L(E, s)L(E^D, s)$  at the centre, and so falls silent above order one.

A low-order witness is the rank identity on every curve of order at most one. The act supplies one (act\_gives\_low\_witness): the act gives the witness. A frame carries one and not the value (low\_witness\_strict): it is strictly smaller. Without one, a curve of order zero carries rank two, as in the frame noLow (low\_witness\_not\_given). The theorem of Gross and Zagier with Kolyvagin (Gross and Zagier 1986; Kolyvagin 1990), with Coates and Wiles in the CM case (Coates and Wiles 1977), carried to every  $E$  over  $\mathbb{Q}$  by modularity (Breuil, Conrad, Diamond and Taylor 2001) and the non-vanishing of a quadratic twist (Bump, Friedberg and Hoffstein 1990; Murty and Murty 1991), has exactly this shape: a witness the act supplies on the low half (act\_gives\_low\_witness), never a premise of the closure; and the low-order witness does not give the full row, a frame carrying one and failing at the leading coefficient (low\_witness\_not\_full).

**The exhibit at the first open order.** On the curve 389a1, of rank two, the points  $(-1, 1)$  and  $(0, 0)$  have,

by the local decomposition, Tate's series at the real place and the denominator at the finite places, canonical heights 0.6866670833 and 0.3270007737 and pairing  $-0.2684780988$ , so the regulator is 0.1524601779. From the Fourier coefficients, with the root number  $+1$  read from the reduction at 389,  $L(E, 1)$  computes to zero within  $2 \times 10^{-14}$  and  $L'(E, 1)$  vanishes by the sign, so the analytic order is two within the tolerance of the script, the number of supplied independent generators, and  $L''(E, 1)/2! = 0.7593165003$ ; and the real period, by the arithmetic-geometric mean, is 4.9804251217; with trivial torsion and Tamagawa product, the analytic order of the Tate-Shafarevich group computes to 1 within  $1.8 \times 10^{-11}$ , an integer square as the formula requires; at 389a1 the order of the Tate-Shafarevich group itself is not known to be finite. The same code passes at 11a1, order zero, to  $6.1 \times 10^{-14}$ , and at 37a1, order one, to  $1.4 \times 10^{-12}$ , orders at most one, where the Tate-Shafarevich group is finite by Kolyvagin (Kolyvagin 1990), before the rank-two figure is read. The figures agree with the tabulated invariants (Cremona 1997). The exhibit is raw data at corroboration grade, outside the kernel; it gives any construction fixed in advance a target, a height determinant of 0.1524601779 on that curve.

#### 14. THE DIVISION AND THE TWO PARTS

For every partition of the curves the value is exactly its two halves (`row_split`). At order one the low half holds and the remainder is not forced (`remainder_not_forced`). The full row has two parts, the rank and the leading coefficient (`two_parts`), and the rank part does not give the coefficient part (`rank_part_not_full`). Read on the full row, the act registers both: the act read on the full row, the full act, is the full value, exactly (`full_act_is_the_full_value`), the root read on the full row is that act and keyed (`root_on_full_row_is_the_full_act`, `root_on_full_row_is_the_full_value`, `full_act_is_keyed`), and from it, by one act, the compiler prints the full value (`bsd_full_from_existence`); its supply is exact (`full_supply_iff`), it is the weakest premise of the full value (`full_act_is_the_weakest_premise`), it carries the act of the rank part (`full_act_gives_rank_act`), and one curve whose coefficient is not the product refutes the full act (`coefficient_refutes`). The full act is strictly stronger than the act: it carries the act, and a frame carries the act and not the full act, so the act does not give the full value (`full_act_strictly_stronger`, `act_does_not_give_full_value`); the root read on the full row grounds itself exactly where the full value holds (`full_row_root_grounds_itself_iff_full_value`). Two supply fields stand in the kernel, supply of `ActualCurves` and supply of `ActualCurvesFull`; the first closes the rank part, the second the full row.

#### 15. RANK AS A DIMENSION COUNT, AND THE RETIRED PREMISE

Over  $\mathbb{F}_2^3$  one nonzero class spans exactly two points, two independent classes four, three independent classes eight, and one or two classes never more than two or four (`one_class_spans_two`, `two_classes_span_four`, `three_classes_span_eight`, `k_classes_bound`); and for every number of classes, the combinations of  $k$  classes number  $2^k$ , fewer than the  $2^r$  points of  $\mathbb{F}_2^r$  whenever  $k < r$  (`compos_length`, `points_length`, `fewer_classes_fall_short`). One class cannot certify rank two; certifying rank  $r$  takes  $r$  classes. The act at order  $r$  is an  $r$ -dimensional registration, the dual of the triaxial count: for one and for two classes, the points they span times the solutions the same number of independent constraints leaves is the eight points of  $\mathbb{F}_2^3$  (`span_and_lock_are_dual`). The rank of a group of rational points is, by definition, the largest number of independent classes in it. Classes are independent when their combinations are pairwise distinct points, the points of their span listed once each (`Independent`, on `distinctB`); three unit classes are independent, and a class listed twice is not (`independence_discriminates`). For a lattice of independent classes over  $\mathbb{F}_2$  the kernel proves that the rank is a dimension count: the combinations of  $k$  independent classes list the  $2^k$  points of their span once each, and independent lattices with equally many points have equally many classes, two to the power being injective, so the rank, the number of independent classes, is read off the number of points of the span (`rank_is_the_dimension_count`, `two_pow_inj`). What stays the reader's identification, declared here and in Section 17, is narrower: that the frame's rank field is the rank of the Mordell-Weil lattice of the curve, read through its classes modulo two with the torsion classes set aside, a cited fact of the subject (Mordell 1922), not proved here.

The author's earlier draft derived the row from one axiom,  $\Omega$ : for every curve and every order  $r \geq 2$ , a family of  $r$  classes produced by a construction fixed in advance, consulting neither the rank nor the Selmer group, with height, descent and Selmer clauses. The kernel types it on exactly the curves it constrains, its fixed-construction clause on every curve of order at least two joined to the value (`OmegaHolds`):  $\Omega$  gives the value (`omega_gives_value`) and is strictly stronger, since a curve of order two carries the value with no construction fixed in advance (`omega_strictly_stronger`). It is retired as a placeholder written before the act was isolated, and the act takes its place at exactly the value's strength.

#### 16. THE RECORD AND THE SEED

No finite record of curves forces the value: for every  $n$  the staged world satisfies the identity on its first  $n$

curves and fails (`finite_record_never_forces`); above order one the staged world of order two (`stagedHigh`) keeps the low-order witness, satisfies the identity on its first  $n$  curves and fails (`finite_record_never_forces_above_one`). A step uniform in the order would force every order (`uniform_step_forces_all`); no cited theorem carries one past order one.

## 17. THE GRADE OF THE CLOSURE, STATED WHOLE

**The root, universal and undeniable in deed:** every denial of it re-enacts it, no outside proof adds to it, and it holds on every frame once any deed occurs (`denial_reenacts_root`, `external_proof_adds_nothing`); for that universality it neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`), and read on the row it is the act, keyed, at premise grade, grounding itself exactly where the value holds (`root_read_on_row_is_keyed`, `row_root_grounds_itself_iff_value`).

**Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels):** the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

**Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (one hundred one theorems):** the seat from first principle and the parity of the order, on every centred expansion and on every frame carrying them; the order unique and the first-order reading silent above order one; the frame, its worlds; existence given and forcing no value; keyless and keyed; the act equal to the value; the rank part from the act; the exclusive gates and the one refuter; the parity bit; the low-order witness supplied by the act and strictly smaller; the division and the two parts; the full row read by the act, the full act, strictly stronger than the act, which does not give the full value; the full value from the full act by one act, its exact supply, its weakest premise, its refuter;  $\Omega$  retired on the curves it constrains; the dimension count over  $\mathbb{F}_2$ , for every number of classes, the rank of a lattice of independent classes read off it, and span dual to lock; the record wall, in the Bool model and on frames, and the bit as the world; the two-point fibre, the prime's shape; the triaxial counts; the finite record, below and above order one, and the uniform step; the closure whole (`bsd_hardenened_closure`); the root satisfiable on every background and on its constructed domain (`root_satisfiable`, `constructed_root_holds`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_on_row_is_keyed`); the act the weakest premise that forces the value

(`act_is_the_weakest_premise`); nothing given refutes the act on every frame, and nothing given forces the value on every frame (`nothing_given_refutes`, `nothing_given_forces`); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall and the finite record in one theorem (`the_lock`); the root grounding itself on the row and on the full row exactly where the value and the full value hold (`row_root_grounds_itself_iff_value`, `full_row_root_grounds_itself_iff_full_value`); the constructed root by its deed, the root of Appendix R repeated verbatim and equal to it, and the lock at that root (`constructed_root_by_its_deed`, `RA0is_the_constructed_root`, `the_lock_on_the_root`).

**The premise ledger.** Proved unconditionally, at theorem grade, on no axiom and no posit: the root (Appendix R). Given and proved satisfiable: the arrow, the freedom bit. Supplied: the act, the field supply of `ActualCurves`, the only premise of the rank part; and the act read on the full row, the full act, the field supply of `ActualCurvesFull`, the only premise of the full value, strictly stronger than the act (`full_act_strictly_stronger`, `act_does_not_give_full_value`). The kernel's other structures are data or the hypotheses of universally quantified theorems; none carries a cited theorem.

**Witnesses, not premises:** modularity, for the seat's application to curves; the height formula, for the first-order reading; the theorem at order at most one, with modularity and the non-vanishing of a quadratic twist for its reach to every curve over  $\mathbb{Q}$ , for the low-order witness.

**Corroboration, outside the kernel:** the exhibit on 389a1 and its calibrations.

**The reader's identification:** the frame with the invariants of elliptic curves over  $\mathbb{Q}$ , and the frame's rank field with the rank of the Mordell–Weil lattice of the curve, read through its classes modulo two with the torsion classes set aside (Mordell 1922); that the rank of a lattice of independent classes over  $\mathbb{F}_2$  is its dimension count is proved, not identified (`rank_is_the_dimension_count`).

## 18. OBJECTIONS, ANSWERED

*The act is only a premise.* It stands on the root read on the row. The root's own undeniability is theorem grade and keyless: every denial re-enacts it and no outside proof adds to it (`denial_reenacts_root`, `external_proof_adds_nothing`), and it holds in both worlds, so it neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`). Read on the row the root is the act, keyed, which decides it (`root_read_on_row_is_keyed`), and it grounds itself exactly where the value holds, on the full row exactly where the full value holds (`row_root_`

grounds\_itself\_iff\_value, full\_row\_root\_grounds\_itself\_iff\_full\_value): the undeniability the act carries is the act's own, at premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

*The act is the conjecture renamed.* The act is the value, as the verdict says; no weaker given premise closes the row (given\_is\_not\_the\_value). The isolation of the least premise is the result.

*Gross-Zagier and Kolyvagin are doing the work.* They are not premises of any closing theorem; they are typed as a witness of the act on the low half, supplied by the act and strictly smaller, and not giving the full row (act\_gives\_low\_witness, low\_witness\_strict, low\_witness\_not\_full).

*The parity conjecture is the real content.* The kernel proves the parity of the order from the sign and proves the parity is not the value (minus\_order\_odd, parity\_is\_not\_the\_value).

*The axiom  $\Omega$  was a reasonable route.* It overpaid: strictly stronger than the value (omega\_strictly\_stronger).

*The coefficient sequence is a skeleton, not an L-function.* The seat theorems hold of every centred expansion; that a curve's completed L-function, read by the vanishing of its coefficients, is one is modularity, a witness, and the closure does not use it.

## 19. FALSIFIERS

**F-Cone.** The kernel prints any axiom for any of its one hundred one theorems. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

**F-Premise.** A closing theorem that consumes any field other than the two supply fields, supply of ActualCurves and supply of ActualCurvesFull, and the hypotheses of universally quantified statements.

**F-Curve.** An elliptic curve over  $\mathbb{Q}$  whose rank differs from its order of vanishing at the centre; it refutes the act by counter\_curve\_refutes.

**F-Coefficient.** An elliptic curve over  $\mathbb{Q}$  whose leading coefficient at the centre differs from the arithmetic product; it refutes the full act by coefficient\_refutes, and it is observable only where the order of the Tate-Shafarevich group is known.

## 20. POSITIONING

## 21. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame, and the value neither forced on every frame nor excluded on every frame. It proves that existence read on the row is the value; that the rank part follows from it by one act, the only premise of that part, and the full row from the full act, strictly stronger; that nothing escapes, that one counter curve refutes the act, and that wherever the act fails a counter curve cannot

be excluded; that the sign fixes the parity of the order and the parity is not the value; that every first-order reading falls silent above order one; that the low-order theorem is a witness of the act; and that the higher-cycle axiom overpaid. The verdict, in the words of Section 1, unchanged:

The Birch and Swinnerton-Dyer row is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the calm world and the counter world alike. Existence read on the row, that every elliptic curve that exists registers its rank on the seat, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, the only premise the closure of the rank part admits, and the compiler prints the rank part from it on no axiom; read on the full row, where existence also registers the leading coefficient as the arithmetic product, the act read on the full row, the full act, strictly stronger, is the only premise the full closure admits, and the compiler prints the full value from it the same way. Nothing escapes the act, one curve whose rank differs from its order of vanishing would refute it, and one whose leading coefficient is not the arithmetic product would refute the full act. At order at most one the subject's theorem is a witness of the act; above order one the value stands on the act, and in the leading coefficient on the full act, strictly stronger, the only premise the full closure admits.

## APPENDIX A • RECEIPTS

The kernel, BSD\_Hardened.lean, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021). It carries one hundred one theorems, and every one prints *does not depend on any axioms*, pinned by #guard\_msgs. Its SHA-256 is

```
d52b7a9e5b4f9be4dd02866065e9d56b
6509ddacda4d6340897aa3337ac0c8be
```

The root kernel of Appendix R, T0E\_Zero.lean, compiles with exit 0 and no message on both toolchains; its fifteen theorems print *does not depend on any axioms*, pinned by #guard\_msgs. The Markdown master carries all four kernels and the exhibit script with a one-line extraction command and their manifest. The two further kernels of Appendix R, Triaxial\_Actuation.lean and Root\_Grade\_Ledger.lean, compile with exit 0 and no message on both toolchains; their thirty-two

| Work                                         | Position                            | This paper                                            | Relation |
|----------------------------------------------|-------------------------------------|-------------------------------------------------------|----------|
| Birch and Swinnerton-Dyer 1965               | the question posed                  | closed to one act                                     | extends  |
| Coates and Wiles 1977                        | CM curves, order zero               | part of the low-order witness                         | extends  |
| Gross and Zagier 1986;<br>Kolyvagin 1990     | the identity at order at most one   | typed as a witness the act supplies, strictly smaller | extends  |
| Wiles 1995; Taylor and Wiles 1995; BCDT 2001 | modularity                          | a witness of the seat's application                   | adjacent |
| Cremona 1997                                 | the tabulated curves                | the exhibit's agreement                               | adjacent |
| Wiles 2006                                   | the problem stated                  | the setting of Section 2                              | adjacent |
| Islam 2026a, b, h, i, j                      | rows closed to one act of existence | the drill carried to the last Millennium row          | extends  |

*Table 3 | Positioning. The relation words: extends where the result is carried further, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.*

and eighteen theorems print *does not depend on any axioms*, pinned by #guard\_msgs; their SHA-256 digests, 566615a6b9610fc9... and c18b68b29d2b6be5..., stand in full in the manifest beside the two others and in Appendix R, and the master carries all four kernels.

## APPENDIX B • THE KERNEL

```

/-
  BSD_Hardened.lean • the kernel of the hardened master seed of the
  Birch and Swinnerton-Dyer closure
  (supersedes BSD_Master.lean of APEX-PSP-BSD-MASTER-01)

First principle, premise-free except existence and freedom. Every
theorem of this file is on no
axiom at all: no propext, no Quot.sound, no Classical.choice. No
structure of the closure carries a
cited theorem as a field. The only premise any closing theorem
consumes is the act, existence read
on the row; the root is satisfiable on every background and proved
on its constructed domain
(root_satisfiable, constructed_root_holds), and the arrow and the
freedom bit are given and proved
satisfiable. The cited results of the subject enter only as
witnesses typed against the act, supplied
by it and strictly smaller, never as premises.
I   The seat from first principle: a centred expansion with a
    sign; a minus sign silences every
    even coefficient, a plus sign every odd one, so the sign
    fixes the parity of the order, and
    a minus sign forces vanishing at the centre.
II  The order and its first reading: below the order every
    coefficient vanishes by definition,
    so above order one the first-order reading is silent, and at
    order one it speaks.
III The frame of curves, the value, the worlds; existence as
    given forces no value.
IV  The act: every curve that exists registers its rank on the
    seat; it is the value; one act
    closes it; nothing escapes; one curve refutes; nothing given
    refutes the act or forces the value.
V   The parity bit: one bit, and not the value; on a frame
    carrying centred expansions the sign
    fixes the parity of each curve's order.
VI  The low-order witness typed: any witness of the rank
    identity at order at most one is
    supplied by the act and is strictly smaller.
VII The division at order one, and the two parts; a low-order
    witness does not give the full
    value; the full row: read on the full row the act registers
    the rank on the seat and the
    leading coefficient as the arithmetic product; it is the
    full value; one act closes it; one
    curve refutes it; the full act is strictly stronger than the
    act, which does not give the
    full value.
VIII Keyless and keyed.
IX  Freedom, the prime's shape, the triaxial count; the record
    wall in the Bool model and on
    frames; the bit is the world.
X   The record and the seed: a finite record never forces the
    value, nor above order one with
    the low-order witness in place.
XI  The retired premise  $\Omega$ , typed on the curves of order at least
    two.

```

```

XII Rank is a dimension count over GF(2): in GF(2)^3 by
    decision, span and lock dual for one and
    two classes, and for every number of classes, k classes
    reaching at most 2^k of the 2^r
    points of GF(2)^r; and the count returns the rank:
    independent classes list the points of
    their span once each, and independent lattices with equally
    many points have equally many
    classes (rank_is_the_dimension_count).
XIII The hardened closure, whole.
ROOT The root, undeniable in deed, and the lock: the root holds
    by its deed and forces neither the
    value nor its failure; read on the row it is the act, and it
    grounds itself on the row or the
    full row exactly where the value or the full value holds;
    one theorem on no axiom binds the root,
    its readings, the act, the value, their keying, the closing
    theorem, the record wall and the
    finite record (the_lock); the constructed root by its deed,
    and the root of Appendix R repeated
    verbatim, which is the constructed root and carries the lock.
-/

namespace BSDHard

/-! ## I • the seat from first principle -/

/-- An integer equal to its own negative is zero. -/
theorem self_neg_zero : ∀ v : Int, v = -v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (_ + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- Evenness, by recursion. -/
def evenB : Nat → Bool
| 0 => true
| n + 1 => !evenB n

/-- A centred expansion with a sign: the coefficients of a function
about the centre, and the
reflection law of a functional equation about the centre with that
sign, which negates the k-th
coefficient exactly when its parity disagrees with the sign: an even
coefficient under a minus sign,
an odd one under a plus sign. -/
structure Centred where
  a      : Nat → Int
  minus  : Bool
  reflect : ∀ k, (evenB k == minus) = true → a k = -(a k)

/-- A minus sign silences every even coefficient. -/
theorem minus_silences_even (C : Centred) (hm : C.minus = true) (k :
  Nat) (hk : evenB k = true) :
  C.a k = 0 :=
  self_neg_zero _ (C.reflect k (by rw [hm, hk]; rfl))

/-- A plus sign silences every odd coefficient. -/
theorem plus_silences_odd (C : Centred) (hm : C.minus = false) (k :
  Nat) (hk : evenB k = false) :
  C.a k = 0 :=
  self_neg_zero _ (C.reflect k (by rw [hm, hk]; rfl))

/-- THE SEAT: a minus sign forces vanishing at the centre. -/
theorem minus_sign_forces_vanishing (C : Centred) (hm : C.minus =
  true) : C.a 0 = 0 :=
  minus_silences_even C hm 0 rfl

```

```

/-- The order of a coefficient sequence: the first index whose
    coefficient is nonzero. -/
def IsOrder (a : Nat → Int) (o : Nat) : Prop := a o ≠ 0 ∧ ∀ j, j < o
→ a j = 0

/-- THE SIGN FIXES THE PARITY OF THE ORDER: under a minus sign the
    order is odd. -/
theorem minus_order_odd (C : Centred) (hm : C.minus = true) (o :
    Nat) (ho : IsOrder C.a o) :
    evenB o = false :=
    match h : evenB o with
    | true => absurd (minus_silences_even C hm o h) ho.1
    | false => rfl

/-- Under a plus sign the order is even. -/
theorem plus_order_even (C : Centred) (hm : C.minus = false) (o :
    Nat) (ho : IsOrder C.a o) :
    evenB o = true :=
    match h : evenB o with
    | false => absurd (plus_silences_odd C hm o h) ho.1
    | true => rfl

/-! ## II · the order and its first reading -/

/-- THE FIRST-ORDER READING IS SILENT ABOVE ORDER ONE, by the
    definition of the order. -/
theorem first_reading_silent_above_one (a : Nat → Int) (o : Nat) (ho :
    IsOrder a o) (h : 2 ≤ o) :
    a 1 = 0 :=
    ho.2 1 h

/-- At order one the first-order reading speaks. -/
theorem first_reading_speaks_at_one (a : Nat → Int) (ho : IsOrder a
    1) : a 1 ≠ 0 := ho.1

/-- The order is unique. -/
theorem order_unique (a : Nat → Int) (o p : Nat) (ho : IsOrder a o)
    (hp : IsOrder a p) : o = p :=
    match Nat.lt_or_ge o p with
    | Or.inl hlt => absurd (hp.2 o hlt) ho.1
    | Or.inr hge => match Nat.eq_or_lt_of_le hge with
    | Or.inl he => he.symm
    | Or.inr hlt => absurd (ho.2 p hlt) hp.1

/-! ## III · the frame, and existence as given -/

/-- A frame of elliptic curves: each with its rank, its order of
    vanishing at the centre, and
whether the leading coefficient equals the arithmetic product. -/
structure Frame where
    D : Type
    rank : D → Nat
    order : D → Nat
    coeff : D → Bool

/-- The value of the rank part: the rank is the order of vanishing,
    for every curve. -/
def Value (F : Frame) : Prop := ∀ d, F.rank d = F.order d

def calm : Frame := (Unit, fun _ => 1, fun _ => 1, fun _ => true)
/-- The counter world: a curve of rank zero whose order of vanishing
    is two. -/
def counter : Frame := (Unit, fun _ => 0, fun _ => 2, fun _ => true)

theorem calm_value : Value calm := fun _ => rfl
theorem counter_fails : ¬ Value counter := fun h => Nat.noConfusion
    (h ())

/-! ### existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e =>
    nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U →
    Int), Root U ΔE → S) : S :=
    h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => (id, fun _ => rfl)

theorem freedom_given : ∀ w : Bool, (!w) ≠ w

```

```

| true => Bool.noConfusion
| false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) : ¬ ∀ F : Frame, (P ↔
    Value F) :=
    fun h => counter_fails ((h counter).mpr ((h calm).mpr calm_value))

theorem arrow_forces_nothing : ¬ ∀ F : Frame, Arrow → Value F :=
    fun h => counter_fails (h counter arrow_given)

theorem given_in_both_worlds : (Arrow ∧ Value calm) ∧ (Arrow ∧ ¬
    Value counter) :=
    ((arrow_given, calm_value), (arrow_given, counter_fails))

/-! ## IV · existence read on the row: the act -/

/-- Existence read on the row: every curve that exists registers its
    rank on the seat. -/
def Registered (F : Frame) : Prop := ∀ d, F.rank d = F.order d

theorem act_is_the_value (F : Frame) : Registered F ↔ Value F :=
    (fun h d => h d, fun h d => h d)

theorem act_is_keyed : Registered calm ∧ ¬ Registered counter :=
    (calm_value, counter_fails)

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero
    where it fails. -/
def indicator {p : Prop} : Decidable p → Int
    | isTrue _ => 1
    | isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds.
    -/
theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h
    ↔ p :=
    match h with
    | isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)
    | isFalse hn => (fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by
        decide), fun hp => absurd hp hn)

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable (∃ n, o = some n) :=
    match o with
    | some n => isTrue (n, rfl)
    | none => isFalse (fun (_, h) => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, `Root` itself,
    with the row's existents, its curves,
and the row's actuation, one where the curve registers its rank at
    its order and zero where it does not. -/
def RootOnRow (F : Frame) : Prop :=
    Root F.D (fun d => indicator (Nat.decEq (F.rank d) (F.order d)))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
    actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _
    => show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent,
    whose actuation is one. A theorem
with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
    root_satisfiable Unit

/-! THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F ↔
    Registered F :=
    (fun h d => (indicator_pos _).mpr (h d), fun h d => (indicator_pos
        _).mpr (h d))

/-! THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (F : Frame) : RootOnRow F ↔ Value F
    :=
    (fun h => (act_is_the_value F).mpr ((root_on_row_is_the_act F).mpr
        h),
    fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr
        h))

/-! THE ROOT READ ON THE ROW IS KEYED: it holds where the value
    holds and fails where the value fails. -/

```

```

theorem root_on_row_is_keyed : RootOnRow calm  $\wedge$   $\neg$  RootOnRow counter
:=
  ((root_on_row_is_the_value calm).mpr calm_value, fun h =>
    counter_fails ((root_on_row_is_the_value counter).mp h))

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on
    every frame, and every premise that forces
    the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
  ( $\forall$  F : Frame, Registered F  $\rightarrow$  Value F)  $\wedge$ 
   $\forall$  Q : Frame  $\rightarrow$  Prop, ( $\forall$  F : Frame, Q F  $\rightarrow$  Value F)  $\rightarrow$   $\forall$  F : Frame, Q
  F  $\rightarrow$  Registered F :=
  (fun F h => (act_is_the_value F).mp h, fun _ hQ F hq =>
    (act_is_the_value F).mpr (hq F hq))

/-- EXISTENCE AS GIVEN HOLDS IN BOTH WORLDS: the root satisfiable on
    every background, the arrow and the
    freedom bit hold in the world where the value holds and in the world
    where it fails. -/
theorem existence_as_given_in_both_worlds :
  (( $\forall$  U : Type, Root U (fun _ => 1))  $\wedge$  Arrow  $\wedge$  ( $\forall$  w : Bool, (!w)  $\neq$ 
  w))  $\wedge$  Value calm  $\wedge$   $\neg$  Value counter :=
  ((root_satisfiable, arrow_given, freedom_given), calm_value,
  counter_fails)

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes
    the act on every frame. -/
theorem nothing_given_refutes :  $\forall$  P : Prop, P  $\rightarrow$   $\neg$   $\forall$  F : Frame, P  $\rightarrow$   $\neg$ 
  Registered F :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces
    the value on every frame. -/
theorem nothing_given_forces :  $\forall$  P : Prop, P  $\rightarrow$   $\neg$   $\forall$  F : Frame, P  $\rightarrow$ 
  Value F :=
  fun _ hp h => counter_fails (h counter hp)

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no
    instance fails, a curve whose rank is not its order. -/
theorem only_refuter (F : Frame) :  $\neg$  Registered F  $\rightarrow$   $\neg$   $\exists$  d, F.rank
  d  $\neq$  F.order d :=
  fun hn hne => hn (fun d => match Nat.decEq (F.rank d) (F.order d)
  with
  | isTrue h => h
  | isFalse h => absurd (d, h) hne)

structure ActualCurves where
  F : Frame
  supply : Registered F

/-- THE RANK PART FROM EXISTENCE, BY ONE ACT. -/
theorem bsd_from_existence (A : ActualCurves) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the
    value holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualCurves // A.F
  = F }  $\leftrightarrow$  Value F :=
  (fun ((A, hA)) => hA  $\triangleright$  bsd_from_existence A,
  fun h => (((F, (act_is_the_value F).mpr h), rfl)))

/-- Every curve lands on exactly one gate: its rank equals its
    order, or not. -/
theorem every_curve_lands (F : Frame) (d : F.D) : F.rank d = F.order
  d  $\vee$  F.rank d  $\neq$  F.order d :=
  match Nat.decEq (F.rank d) (F.order d) with
  | isTrue h => Or.inl h
  | isFalse h => Or.inr h

theorem gates_exclusive (F : Frame) (d : F.D) :  $\neg$  (F.rank d =
  F.order d  $\wedge$  F.rank d  $\neq$  F.order d) :=
  fun (h, h') => h' h

theorem nothing_escapes (A : ActualCurves) :  $\forall$  d, A.F.rank d =
  A.F.order d := A.supply

/-- One curve whose rank differs from its order refutes the act. -/
theorem counter_curve_refutes (F : Frame) (d : F.D) (h : F.rank d  $\neq$ 
  F.order d) :  $\neg$  Registered F :=
  fun hA => h (hA d)

/-! ## V  $\cdot$  the parity bit -/

```

```

def ParityValue (F : Frame) : Prop :=  $\forall$  d, F.rank d % 2 = F.order d
  % 2

theorem value_gives_parity (F : Frame) (h : Value F) : ParityValue F
:=
  fun d => by rw [h d]

/-- THE PARITY IS ONE BIT AND NOT THE VALUE: the counter world keeps
    the parity and fails. -/
theorem parity_is_not_the_value : ParityValue counter  $\wedge$   $\neg$  Value
  counter :=
  (fun _ => show 0 % 2 = 2 % 2 from rfl, counter_fails)

/-- A frame carrying centred expansions: each curve with its centred
    expansion, whose order is the curve's order. -/
structure SignedFrame where
  F : Frame
  L : F.D  $\rightarrow$  Centred
  ord :  $\forall$  d, IsOrder (L d).a (F.order d)

/-- THE SIGN FIXES THE PARITY OF THE CURVE'S ORDER: a curve whose
    expansion carries a minus sign has odd order. -/
theorem sign_fixes_curve_parity (S : SignedFrame) (d : S.F.D) (hm :
  (S.L d).minus = true) :
  evenB (S.F.order d) = false :=
  minus_order_odd (S.L d) hm (S.F.order d) (S.ord d)

/-! ## VI  $\cdot$  the low-order witness typed -/

/-- A low-order witness: the rank identity on every curve of order
    at most one. The cited theorem
    of the subject at order at most one has this shape; here it is a
    type, not a premise. -/
def LowWitness (F : Frame) : Prop :=  $\forall$  d, F.order d  $\leq$  1  $\rightarrow$  F.rank d =
  F.order d

/-- SUPPLIED BY THE ACT: the act gives a low-order witness. -/
theorem act_gives_low_witness (A : ActualCurves) : LowWitness A.F :=
  fun d _ => A.supply d

/-- STRICTLY SMALLER: a frame carries a low-order witness and not
    the value. -/
theorem low_witness_strict : LowWitness counter  $\wedge$   $\neg$  Value counter :=
  (fun _ h => absurd h (by decide :  $\neg$  ((2 : Nat)  $\leq$  1)),
  counter_fails)

/-- A frame with no low-order witness: a curve of order zero
    carrying rank two. -/
def noLow : Frame := (Unit, fun _ => 2, fun _ => 0, fun _ => true)

theorem low_witness_not_given :  $\neg$  LowWitness noLow := fun h =>
  Nat.noConfusion (h ()) (by decide))

/-! ## VII  $\cdot$  the division and the two parts -/

def ValueOn (F : Frame) (cut : F.D  $\rightarrow$  Bool) (b : Bool) : Prop :=
   $\forall$  d, cut d = b  $\rightarrow$  F.rank d = F.order d

theorem row_split (F : Frame) (cut : F.D  $\rightarrow$  Bool) :
  Value F  $\leftrightarrow$  ValueOn F cut true  $\wedge$  ValueOn F cut false :=
  (fun h => (fun d _ => h d, fun d _ => h d),
  fun (ht, hf) d => match hc : cut d with
  | true => ht d hc
  | false => hf d hc)

/-- At order one the proved half holds and the remainder fails, in
    the counter world. -/
theorem remainder_not_forced :
  ValueOn counter (fun d => Nat.ble (counter.order d) 1) true  $\wedge$ 
   $\neg$  ValueOn counter (fun d => Nat.ble (counter.order d) 1) false :=
  (fun _ h => absurd h (by decide :  $\neg$  (Nat.ble 2 1 = true))),
  fun h => counter_fails (fun d => h d rfl))

/-- The full value: the rank part and the leading-coefficient part.
    -/
def FullValue (F : Frame) : Prop := Value F  $\wedge$   $\forall$  d, F.coeff d = true

def coeffCounter : Frame := (Unit, fun _ => 1, fun _ => 1, fun _ =>
  false)

/-- THE RANK PART DOES NOT GIVE THE LEADING COEFFICIENT. -/
theorem rank_part_not_full : Value coeffCounter  $\wedge$   $\neg$  FullValue
  coeffCounter :=
  (fun _ => rfl, fun (_, h) => Bool.noConfusion (h ()))

```

```

theorem two_parts (F : Frame) : FullValue F  $\leftrightarrow$  Value F  $\wedge \forall d, F.coeff$ 
  d = true := Iff.rfl

/-- A LOW-ORDER WITNESS DOES NOT GIVE THE FULL VALUE: a frame
  carries one and fails at the leading coefficient. -/
theorem low_witness_not_full : LowWitness coeffCounter  $\wedge \neg$  FullValue
  coeffCounter :=
  (fun _ => rfl, rank_part_not_full.2)

/-! ### the full row -/

/-- Existence read on the full row: every curve that exists
  registers its rank on the seat and its leading
  coefficient as the arithmetic product. -/
def RegisteredFull (F : Frame) : Prop :=  $\forall d, F.rank\ d = F.order\ d \wedge$ 
  F.coeff d = true

/-- THE FULL ACT IS THE FULL VALUE. -/
theorem full_act_is_the_full_value (F : Frame) : RegisteredFull F  $\leftrightarrow$ 
  FullValue F :=
  (fun h => (fun d => (h d).1, fun d => (h d).2), fun h d => (h.1 d,
    h.2 d))

/-- The full act carries the act of the rank part. -/
theorem full_act_gives_rank_act (F : Frame) (h : RegisteredFull F) :
  Registered F := fun d => (h d).1

/-- THE FULL ACT IS KEYED: it holds in the calm world and fails
  where the coefficient fails and where the rank fails. -/
theorem full_act_is_keyed : RegisteredFull calm  $\wedge \neg$  RegisteredFull
  coeffCounter  $\wedge \neg$  RegisteredFull counter :=
  (fun _ => (rfl, rfl), fun h => Bool.noConfusion (h ()).2, fun h =>
    Nat.noConfusion (h ()).1)

/-- THE ROOT READ ON THE FULL ROW: `Root` itself, with the row's
  curves as existents and actuation one where the curve
  registers its rank at its order and its leading coefficient, zero
  where it does not. -/
def RootOnFullRow (F : Frame) : Prop :=
  Root F.D (fun d => indicator (@instDecidableAnd _ _ (Nat.decEq
    (F.rank d) (F.order d)) (instDecidableEqBool (F.coeff d) true)))

/-- THE ROOT READ ON THE FULL ROW IS THE FULL ACT. -/
theorem root_on_full_row_is_the_full_act (F : Frame) : RootOnFullRow
  F  $\leftrightarrow$  RegisteredFull F :=
  (fun h d => (indicator_pos _).mpr (h d), fun h d => (indicator_pos
    _).mpr (h d))

/-- THE ROOT READ ON THE FULL ROW IS THE FULL VALUE, AND KEYED. -/
theorem root_on_full_row_is_the_full_value :
  ( $\forall F : Frame, RootOnFullRow F \leftrightarrow FullValue F$ )  $\wedge$  RootOnFullRow
  calm  $\wedge \neg$  RootOnFullRow coeffCounter :=
  (fun F => (root_on_full_row_is_the_full_act F).trans
    (full_act_is_the_full_value F),
    (root_on_full_row_is_the_full_act calm).mpr full_act_is_keyed.1,
    fun h => full_act_is_keyed.2.1 ((root_on_full_row_is_the_full_act
      coeffCounter).mpr h))

/-- The actual curves, read on the full row. -/
structure ActualCurvesFull where
  F : Frame
  supply : RegisteredFull F

/-- THE FULL VALUE FROM EXISTENCE, BY ONE ACT: the rank part and the
  leading coefficient. -/
theorem bsd_full_from_existence (A : ActualCurvesFull) : FullValue
  A.F :=
  (full_act_is_the_full_value A.F).mpr A.supply

/-- THE FULL SUPPLY IS EXACT: a full act exists on a frame exactly
  when the full value holds there. -/
theorem full_supply_iff (F : Frame) : Nonempty { A :
  ActualCurvesFull // A.F = F }  $\leftrightarrow$  FullValue F :=
  (fun (A, hA) => hA  $\triangleright$  bsd_full_from_existence A,
    fun h => ((F, (full_act_is_the_full_value F).mpr h), rfl))

/-- THE FULL ACT IS THE WEAKEST PREMISE OF THE FULL VALUE. -/
theorem full_act_is_the_weakest_premise :
  ( $\forall F : Frame, RegisteredFull F \leftrightarrow FullValue F$ )  $\wedge$ 
   $\forall Q : Frame \rightarrow Prop, (\forall F : Frame, Q F \rightarrow FullValue F) \rightarrow \forall F :
    Frame, Q F \rightarrow RegisteredFull F :=
  (fun F h => (full_act_is_the_full_value F).mpr h, fun _ hQ F hq =>
    (full_act_is_the_full_value F).mpr (hQ F hq))$ 
```

```

/-- One curve whose leading coefficient is not the arithmetic
  product refutes the full act. -/
theorem coefficient_refutes (F : Frame) (d : F.D) (h : F.coeff d =
  false) :  $\neg$  RegisteredFull F :=
  fun hA => Bool.noConfusion (h.symm.trans (hA d).2)

/-- THE FULL ACT IS STRICTLY STRONGER THAN THE ACT: it carries the
  act, and a frame carries the act and not the full act. -/
theorem full_act_strictly_stronger :
  ( $\forall F : Frame, RegisteredFull F \rightarrow Registered F$ )  $\wedge$  Registered
  coeffCounter  $\wedge \neg$  RegisteredFull coeffCounter :=
  (full_act_gives_rank_act, rank_part_not_full.1,
    full_act_is_keyed.2.1)

/-- THE ACT DOES NOT GIVE THE FULL VALUE: on some frame the act
  holds and the full value fails. -/
theorem act_does_not_give_full_value :  $\neg \forall F : Frame, Registered F \rightarrow$ 
  FullValue F :=
  fun h => rank_part_not_full.2 (h coeffCounter
    rank_part_not_full.1)

/-! ## VIII · keyless and keyed -/

def Keyed (P : Frame  $\rightarrow$  Prop) : Prop :=  $\exists F, \neg P F$ 

theorem value_is_keyed : Keyed Value := (counter, counter_fails)

theorem no_keyless_statement_is_the_act (P : Prop) :  $\neg \forall F : Frame,$ 
  (P  $\leftrightarrow$  Registered F) :=
  fun h => given_is_not_the_value P (fun F => (h F).trans
    (act_is_the_value F))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e)  $\wedge \neg$ 
  Value counter :=
  (root_given, counter_fails)

/-! ## IX · freedom, the prime, the triaxial count -/

/-- The kinetic record, the Bool model: a record of the free bit
  that reads the same on both of its values. -/
def kineticRecord (_w : Bool) : Nat := 0

/-- THE RECORD WALL, the Bool model: no reading of the kinetic
  record returns the bit. -/
theorem record_wall :  $\neg \exists g : Nat \rightarrow Bool, \forall w, g$  (kineticRecord w) =
  w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/-- THE RECORD WALL ON FRAMES: a record that reads the calm world
  and the counter world alike has no reading that
  is the value on every frame. -/
theorem record_wall_on_frames {Rec : Type} (rec : Frame  $\rightarrow$  Rec) (h :
  rec calm = rec counter) (g : Rec  $\rightarrow$  Prop) :
   $\neg \forall F : Frame, (g (rec F) \leftrightarrow Value F) :=$ 
  fun hall => counter_fails ((hall counter).mpr (h  $\triangleright$  (hall calm).mpr
    calm_value))

/-- The two worlds, indexed by the bit: true gives the calm world,
  false the counter world. -/
def world : Bool  $\rightarrow$  Frame
| true => calm
| false => counter

/-- THE BIT IS THE WORLD: the value holds in the world of a bit
  exactly when the bit is true. -/
theorem the_bit_is_the_world :  $\forall w : Bool, (Value (world w) \rightarrow w =$ 
  true)  $\wedge (w = true \rightarrow Value (world w)) :=$ 
  fun w => match w with
  | true => (fun _ => rfl, fun _ => calm_value)
  | false => (fun h => absurd h counter_fails, fun h =>
    Bool.noConfusion h)

theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord
  w == kineticRecord true)).length = 2 := by
  decide

def mulFibre (n : Nat) : List (Nat  $\times$  Nat) :=
  (List.range (n + 1)).flatMap fun a =>
    (List.range (n + 1)).filterMap fun b => if a * b = n then some
      (a, b) else none

theorem prime_shape :

```

```

mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1
!= p.2) = true := by
decide

/-! ### the triaxial count -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
[false, true].flatMap fun a => [false, true].flatMap fun b =>
[false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r
!= (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
(cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
(xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
r3 =>
r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
[false, true].all (fun t3 =>
solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
by decide

/-! ## X · the record and the seed -/

def staged (n : Nat) : Frame := (Nat, fun d => if d < n then 1 else
0, fun _ => 1, fun _ => true)

theorem finite_record_never_forces (n : Nat) :
(∀ d, d < n → (staged n).rank d = (staged n).order d) ∧ ¬ Value
(staged n) :=
(fun d hd => by show (if d < n then 1 else 0) = 1; rw [if_pos hd],
fun h => by
have e : (staged n).rank n = 0 := by
show (if n < n then 1 else 0) = 0
rw [if_neg (Nat.lt_irrefl n)]
have k := h n
rw [e] at k
exact Nat.noConfusion k)

/-- A staged frame above order one: every curve has order two, rank
two below the stage and rank zero from it on. -/
def stagedHigh (n : Nat) : Frame := (Nat, fun d => if d < n then 2
else 0, fun _ => 2, fun _ => true)

/-- A FINITE RECORD NEVER FORCES THE VALUE ABOVE ORDER ONE: the
low-order witness holds and the record agrees below the
stage, and the value fails. -/
theorem finite_record_never_forces_above_one (n : Nat) :
LowWitness (stagedHigh n) ∧ (∀ d, d < n → (stagedHigh n).rank d
= (stagedHigh n).order d) ∧
¬ Value (stagedHigh n) :=
(fun _ h => absurd h (by decide : ¬ ((2 : Nat) ≤ 1)),
fun d hd => by show (if d < n then 2 else 0) = 2; rw [if_pos hd],
fun h => by
have e : (stagedHigh n).rank n = 0 := by
show (if n < n then 2 else 0) = 0
rw [if_neg (Nat.lt_irrefl n)]
have k := h n
rw [e] at k
exact Nat.noConfusion k)

structure Ladder where
Decided : Nat → Prop
base : Decided 0
mono : ∀ r s, Decided s → r ≤ s → Decided r
δ : Nat → Nat
step : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) :
∀ r, L.Decided r
| 0 => L.base
| r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r
(uniform_step_forces_all L hδ r))
(Nat.add_le_add_left (hδ r) r)

/-! ## XI · the retired premise Ω -/

/-- A frame with its provenance record: for each curve, whether a
construction fixed in advance,
consulting neither the rank nor the Selmer group, produces its
family of classes. -/
structure Provenanced where
F : Frame
fixed : F.D → Bool

/-- The higher-cycle axiom Ω of the earlier draft, on the curves it
constrains: every curve of order at least two
has its family from a construction fixed in advance, and the
family's height determinant, descent and Selmer clauses
give the rank identity. -/
def OmegaHolds (P : Provenanced) : Prop := (∀ d, 2 ≤ P.F.order d →
P.fixed d = true) ∧ Value P.F

theorem omega_gives_value (P : Provenanced) (h : OmegaHolds P) :
Value P.F := h.2

/-- A world of one curve of rank two at order two. -/
def calmTwo : Frame := (Unit, fun _ => 2, fun _ => 2, fun _ => true)

/-- Its curve of order two has no construction fixed in advance. -/
def unprovenanced : Provenanced := (calmTwo, fun _ => false)

/-- THE AXIOM Ω IS RETIRED: it gives the value and is strictly
stronger; a curve of order two carries the value with
no construction fixed in advance, so the value does not hand Ω back.
-/
theorem omega_strictly_stronger : Value unprovenanced.F ∧ ¬
OmegaHolds unprovenanced :=
(fun _ => rfl, fun (h, _) => Bool.noConfusion (h ()) (Nat.le_refl
2)))

/-! ## XII · rank is a dimension count -/

def zero3 : Bool × Bool × Bool := (false, false, false)

/-- The span over GF(2) of a list of vectors in GF(2)^3. -/
def span (vs : List (Bool × Bool × Bool)) : List (Bool × Bool ×
Bool) :=
cube.filter (fun x => vs.foldr (fun v acc => acc ++ acc.map (xor3
v)) [zero3] |>.contains x)

/-- ONE CLASS CANNOT CERTIFY RANK TWO: every nonzero vector spans
exactly two points, rank one. -/
theorem one_class_spans_two : nonzero.all (fun v => (span
[v]).length == 2) = true := by decide

/-- TWO INDEPENDENT CLASSES CERTIFY RANK TWO: they span exactly four
points. -/
theorem two_classes_span_four :
nonzero.all (fun v => nonzero.all (fun w => v == w || (span [v,
w]).length == 4)) = true := by
decide

/-- THREE INDEPENDENT CLASSES CERTIFY RANK THREE: they span all
eight points. -/
theorem three_classes_span_eight :
nonzero.all (fun u => nonzero.all (fun v => nonzero.all (fun w =>
u == v || w == u || w == v || w == xor3 u v || (span [u, v,
w]).length == 8))) = true := by
decide

/-- k classes span at most 2^k points: the certified rank is at most
the number of classes. -/
theorem k_classes_bound :
(span [] ).length = 1 ∧ nonzero.all (fun v => (span [v]).length
≤ 2) = true ∧
nonzero.all (fun v => nonzero.all (fun w => (span [v, w]).length
≤ 4)) = true := by
decide

/-- SPAN AND LOCK ARE DUAL: for one and for two classes, the points
k independent classes span times the solutions

```

```

left by k independent constraints is the eight points of GF(2)^3. -/
theorem span_and_lock_are_dual :
  nonzero.all (fun v => nonzero.all (fun r => [false, true].all
    (fun t =>
      (span [v]).length * solCount [(r, t)] == 8))) = true ∧
  nonzero.all (fun v => nonzero.all (fun w => v == w ||
    nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
      [false, true].all (fun t1 => [false, true].all (fun t2 =>
        (span [v, w]).length * solCount [(r1, t1), (r2, t2)] ==
          8)))))) = true := by
  decide

/-! ### for every number of classes -/

/-- A vector over GF(2) of any length, added coordinatewise. -/
def xorV : List Bool → List Bool → List Bool
| a :: as, b :: bs => xor a b :: xorV as bs
| _, _ => []

/-- Every combination over GF(2) of a list of classes, listed once
per subset of the list. -/
def combos (zero : List Bool) : List (List Bool) → List (List Bool)
| [] => [zero]
| v :: vs => combos zero vs ++ (combos zero vs).map (xorV v)

theorem len_app {α : Type} : ∀ (a b : List α), (a ++ b).length =
  a.length + b.length
| [], b => (Nat.zero_add b.length).symm
| _ :: a, b => (congrArg Nat.succ (len_app a b)).trans
  (Nat.succ_add a.length b.length).symm

theorem add_congr {a b c d : Nat} (h1 : a = b) (h2 : c = d) : a + c =
  b + d := h1 ▸ h2 ▸ rfl

theorem len_map {α β : Type} (f : α → β) : ∀ (a : List α), (a.map
  f).length = a.length
| [] => rfl
| _ :: a => congrArg Nat.succ (len_map f a)

/-- K CLASSES REACH AT MOST 2^K POINTS: the combinations of k
classes are 2^k in number, for every k. -/
theorem combos_length (zero : List Bool) : ∀ vs : List (List Bool),
  (combos zero vs).length = 2 ^ vs.length
| [] => rfl
| v :: vs => ((len_app _ _).trans (congrArg (fun m => (combos zero
  vs).length + m) (len_map (xorV v) _))).trans
  ((congrArg (fun m => m + m) (combos_length zero vs)).trans
    (Nat.mul_two (2 ^ vs.length)).symm)

/-- All the points of GF(2)^r, listed. -/
def points : Nat → List (List Bool)
| 0 => [[]]
| r + 1 => (points r).map (List.cons false) ++ (points r).map
  (List.cons true)

/-- GF(2)^r has 2^r points. -/
theorem points_length : ∀ r : Nat, (points r).length = 2 ^ r
| 0 => rfl
| r + 1 => ((len_app _ _).trans (add_congr ((len_map _ _).trans
  (points_length r))
  ((len_map _ _).trans (points_length r)))).trans (Nat.mul_two
  (2 ^ r)).symm

theorem two_pow_le : ∀ {k r : Nat}, k ≤ r → 2 ^ k ≤ 2 ^ r
| _, 0, h => Nat.le_of_eq (congrArg (2 ^ ·) (Nat.le_zero.mp h))
| _, r + 1, h => match Nat.eq_or_lt_of_le h with
| Or.inl e => Nat.le_of_eq (congrArg (2 ^ ·) e)
| Or.inr l => Nat.le_trans (two_pow_le (Nat.le_of_lt_succ l))
  (Nat.le_trans (Nat.le_add_right (2 ^ r) (2 ^ r))
    (Nat.le_of_eq (Nat.mul_two (2 ^ r)).symm))

/-- FEWER CLASSES FALL SHORT: for every rank r and every k below it,
k classes reach fewer than the 2^k points of
GF(2)^r; certifying rank r takes r classes. -/
theorem fewer_classes_fall_short (zero : List Bool) (vs : List (List
  Bool)) (r : Nat) (h : vs.length < r) :
  (combos zero vs).length < (points r).length :=
  match r, h with
| r + 1, h => (combos_length zero vs).symm ▸ (points_length (r +
  1)).symm ▸
  Nat.lt_of_le_of_lt (two_pow_le (Nat.le_of_lt_succ h))
    (Nat.lt_of_lt_of_le (Nat.lt_add_of_pos_right
      (Nat.two_pow_pos r)) (Nat.le_of_eq (Nat.mul_two (2 ^ r)).symm))

```

```

full_act_is_the_full_value, bsd_full_from_existence,
value_is_keyed)

/-! ## The root, undeniable in deed -/

/-- A self-grounding root, the structure of Appendix R in the same
words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding
root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof
of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) → R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed
in a frame where the value holds
and in one where it fails, so it forces neither the value nor its
failure on every frame. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
  SelfGrounding R) :
  R ∧ Value calm ∧ ~ Value counter ∧ ~ (∀ F : Frame, R → Value F) ∧
  ~ (∀ F : Frame, R → ~ Value F) :=
  (G.instances G.anAct, calm_value, counter_fails, fun h =>
  counter_fails (h counter (G.instances G.anAct)),
  fun h => h calm (G.instances G.anAct) calm_value)

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read
uniformly on the frames it gives no value;
read on the row, with the row's existents and the row's actuation,
it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R ∧ ~ (∀ F : Frame, R → Value F) ∧ (∀ F : Frame, RootOnRow F →
  Value F) ∧ (RootOnRow calm ∧ ~ RootOnRow counter) :=
  (G.instances G.anAct, fun h => counter_fails (h counter
  (G.instances G.anAct)), root_on_row_is_the_value,
  root_on_row_is_keyed)

/-- THE ROOT ON THE ROW GROUNDS ITSELF EXACTLY WHERE THE VALUE
HOLDS: a self-grounding of the root read on the row
exists on a frame exactly when the value holds there. -/
theorem row_root_grounds_itself_iff_value (F : Frame) : Nonempty
  (SelfGrounding (RootOnRow F)) → Value F :=
  (fun (G) => (root_on_row_is_the_value F).mpr (G.instances G.anAct),
  fun h => ((Unit, ()), fun _ => (root_on_row_is_the_value F).mpr
  h)))

/-- THE ROOT ON THE FULL ROW GROUNDS ITSELF EXACTLY WHERE THE FULL
VALUE HOLDS: a self-grounding of the root read on
the full row exists on a frame exactly when the full value holds
there. -/
theorem full_row_root_grounds_itself_iff_full_value (F : Frame) :
  Nonempty (SelfGrounding (RootOnFullRow F)) → FullValue F :=
  (fun (G) => (root_on_full_row_is_the_full_value.1 F).mpr
  (G.instances G.anAct),
  fun h => ((Unit, ()), fun _ =>
  (root_on_full_row_is_the_full_value.1 F).mpr h)))

/-- THE LOCK, one theorem on no axiom: the constructed root holds
with no hypothesis; every self-grounding
root holds by its deed; no given statement is the value on every
frame; the root read uniformly on the frames
gives no value; read on the row the root is the act, the act is the
value exactly, and both are keyed; the
closing theorem gives the value on every actual frame; no reading of
the kinetic record returns the bit; and
no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  Root Unit (fun _ => 1) ∧
  R ∧
  (∀ P : Prop, ~ ∀ F : Frame, (P → Value F)) ∧
  ~ (∀ F : Frame, R → Value F) ∧
  (∀ F : Frame, RootOnRow F → Registered F) ∧

```

```

  (∀ F : Frame, Registered F → Value F) ∧
  (RootOnRow calm ∧ ~ RootOnRow counter) ∧
  (Registered calm ∧ ~ Registered counter) ∧
  (∀ A : ActualCurves, Value A.F) ∧
  (~ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
  (∀ n : Nat, (∀ d, d < n → (staged n).rank d = (staged n).order
  d) ∧ ~ Value (staged n)) :=
  (constructed_root_holds, G.instances G.anAct,
  given_is_not_the_value, (root_read_on_row_is_keyed G).2.1,
  root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
  act_is_keyed, bsd_from_existence, record_wall,
  finite_record_never_forces)

/-! ### the constructed root by its deed -/

/-- The deed of the constructed root: one act, the unit, instanting
the root on one existent of actuation one. -/
def rootDeed : SelfGrounding (Root Unit (fun _ => 1)) := (Unit, ()),
  fun _ => constructed_root_holds)

/-- THE CONSTRUCTED ROOT HOLDS BY ITS DEED AND FORCES NO VALUE: its
deed instances it, and it gives no value on every
frame. -/
theorem constructed_root_by_its_deed : Root Unit (fun _ => 1) ∧ ~ (
  F : Frame, Root Unit (fun _ => 1) → Value F) :=
  (denial_reenacts_root rootDeed rootDeed.anAct,
  (undeniable_root_forces_no_value rootDeed).2.2.2.1)

/-- The actuation of Appendix R, repeated verbatim: one at the
single existent. -/
def ΔE₀ : Unit → Int := fun _ => 1

/-- The root of Appendix R, repeated verbatim: every existent
actuates. -/
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The self-grounding of Appendix R, repeated verbatim: one act,
which instances the root. -/
def ra₀ : SelfGrounding RA₀ := (Unit, ()), fun _ => show (0 : Int)
  < 1 by decide)

/-- THE ROOT OF APPENDIX R IS THE CONSTRUCTED ROOT: the two
statements are one. -/
theorem RA₀_is_the_constructed_root : RA₀ ↔ Root Unit (fun _ => 1)
  := Iff.rfl

/-- THE LOCK ON THE ROOT OF APPENDIX R: the root holds by its deed,
gives no value on every frame, and read on the
row it is the value exactly. -/
theorem the_lock_on_the_root :
  RA₀ ∧ ~ (∀ F : Frame, RA₀ → Value F) ∧ (∀ F : Frame, RootOnRow F
  → Value F) :=
  (ra₀.instances ra₀.anAct, (undeniable_root_forces_no_value
  ra₀).2.2.2.1, root_on_row_is_the_value)

end BSDHard

/-! ## Cones, pinned as printed -/
/-- info: 'BSDHard.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.self_neg_zero
/-- info: 'BSDHard.minus_silences_even' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.minus_silences_even
/-- info: 'BSDHard.plus_silences_odd' does not depend on any axioms
-/-
#guard_msgs in #print axioms BSDHard.plus_silences_odd
/-- info: 'BSDHard.minus_sign_forces_vanishing' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.minus_sign_forces_vanishing
/-- info: 'BSDHard.minus_order_odd' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.minus_order_odd
/-- info: 'BSDHard.plus_order_even' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.plus_order_even
/-- info: 'BSDHard.first_reading_silent_above_one' does not depend
on any axioms -/
#guard_msgs in #print axioms BSDHard.first_reading_silent_above_one
/-- info: 'BSDHard.first_reading_speaks_at_one' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.first_reading_speaks_at_one
/-- info: 'BSDHard.order_unique' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.order_unique
/-- info: 'BSDHard.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.calm_value
/-- info: 'BSDHard.counter_fails' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms BSDHard.counter_fails
/-- info: 'BSDHard.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_given
/-- info: 'BSDHard.root_conservative' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.root_conservative
/-- info: 'BSDHard.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.arrow_given
/-- info: 'BSDHard.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.freedom_given
/-- info: 'BSDHard.given_is_not_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.given_is_not_the_value
/-- info: 'BSDHard.arrow_forces_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.arrow_forces_nothing
/-- info: 'BSDHard.given_in_both_worlds' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.given_in_both_worlds
/-- info: 'BSDHard.act_is_the_value' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.act_is_the_value
/-- info: 'BSDHard.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.act_is_keyed
/-- info: 'BSDHard.bsd_from_existence' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.bsd_from_existence
/-- info: 'BSDHard.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.supply_iff
/-- info: 'BSDHard.every_curve_lands' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.every_curve_lands
/-- info: 'BSDHard.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.gates_exclusive
/-- info: 'BSDHard.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.nothing_escapes
/-- info: 'BSDHard.counter_curve_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.counter_curve_refutes
/-- info: 'BSDHard.value_gives_parity' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.value_gives_parity
/-- info: 'BSDHard.parity_is_not_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.parity_is_not_the_value
/-- info: 'BSDHard.act_gives_low_witness' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.act_gives_low_witness
/-- info: 'BSDHard.low_witness_strict' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.low_witness_strict
/-- info: 'BSDHard.low_witness_not_given' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.low_witness_not_given
/-- info: 'BSDHard.row_split' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.row_split
/-- info: 'BSDHard.remainder_not_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.remainder_not_forced
/-- info: 'BSDHard.rank_part_not_full' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.rank_part_not_full
/-- info: 'BSDHard.two_parts' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_parts
/-- info: 'BSDHard.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.value_is_keyed
/-- info: 'BSDHard.no_keyless_statement_is_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms BSDHard.no_keyless_statement_is_the_act
/-- info: 'BSDHard.pulse_does_not_certify' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.pulse_does_not_certify
/-- info: 'BSDHard.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.record_wall
/-- info: 'BSDHard.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.fibre_is_two
/-- info: 'BSDHard.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.prime_shape
/-- info: 'BSDHard.two_axes_leave_two' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.two_axes_leave_two
/-- info: 'BSDHard.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.three_axes_lock_one

```

```

/-- info: 'BSDHard.finite_record_never_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.finite_record_never_forces
/-- info: 'BSDHard.uniform_step_forces_all' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.uniform_step_forces_all
/-- info: 'BSDHard.omega_gives_value' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.omega_gives_value
/-- info: 'BSDHard.omega_strictly_stronger' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.omega_strictly_stronger
/-- info: 'BSDHard.one_class_spans_two' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.one_class_spans_two
/-- info: 'BSDHard.two_classes_span_four' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.two_classes_span_four
/-- info: 'BSDHard.three_classes_span_eight' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.three_classes_span_eight
/-- info: 'BSDHard.k_classes_bound' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.k_classes_bound
/-- info: 'BSDHard.bsd_hardened_closure' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.bsd_hardened_closure
/-- info: 'BSDHard.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.denial_reenacts_root
/-- info: 'BSDHard.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.external_proof_adds_nothing
/-- info: 'BSDHard.undeniable_root_forces_no_value' does not depend
on any axioms -/
#guard_msgs in #print axioms BSDHard.undeniable_root_forces_no_value
/-- info: 'BSDHard.root_read_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.root_read_on_row_is_keyed
/-- info: 'BSDHard.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.the_lock
/-- info: 'BSDHard.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.indicator_pos
/-- info: 'BSDHard.root_satisfiable' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.root_satisfiable
/-- info: 'BSDHard.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.constructed_root_holds
/-- info: 'BSDHard.root_on_row_is_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.root_on_row_is_the_act
/-- info: 'BSDHard.root_on_row_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.root_on_row_is_the_value
/-- info: 'BSDHard.root_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.root_on_row_is_keyed
/-- info: 'BSDHard.act_is_the_weakest_premise' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.act_is_the_weakest_premise
/-- info: 'BSDHard.existence_as_given_in_both_worlds' does not
depend on any axioms -/
#guard_msgs in #print axioms
BSDHard.existence_as_given_in_both_worlds
/-- info: 'BSDHard.nothing_given_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.nothing_given_refutes
/-- info: 'BSDHard.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.only_refuter
/-- info: 'BSDHard.full_act_is_the_full_value' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_is_the_full_value
/-- info: 'BSDHard.full_act_gives_rank_act' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.full_act_gives_rank_act
/-- info: 'BSDHard.full_act_is_keyed' does not depend on any axioms
-/
#guard_msgs in #print axioms BSDHard.full_act_is_keyed
/-- info: 'BSDHard.root_on_full_row_is_the_full_act' does not depend
on any axioms -/
#guard_msgs in #print axioms
BSDHard.root_on_full_row_is_the_full_act
/-- info: 'BSDHard.root_on_full_row_is_the_full_value' does not
depend on any axioms -/

```

```

#guard_msgs in #print axioms
  BSDHard.root_on_full_row_is_the_full_value
/-- info: 'BSDHard.bsd_full_from_existence' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.bsd_full_from_existence
/-- info: 'BSDHard.full_supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_supply_iff
/-- info: 'BSDHard.full_act_is_the_weakest_premise' does not depend
on any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_is_the_weakest_premise
/-- info: 'BSDHard.coefficient_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.coefficient_refutes
/-- info: 'BSDHard.len_app' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.len_app
/-- info: 'BSDHard.add_congr' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.add_congr
/-- info: 'BSDHard.len_map' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.len_map
/-- info: 'BSDHard.combos_length' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.combos_length
/-- info: 'BSDHard.points_length' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.points_length
/-- info: 'BSDHard.two_pow_le' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_pow_le
/-- info: 'BSDHard.fewer_classes_fall_short' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.fewer_classes_fall_short
/-- info: 'BSDHard.independence_discriminates' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.independence_discriminates
/-- info: 'BSDHard.two_pow_succ_ne_one' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.two_pow_succ_ne_one
/-- info: 'BSDHard.two_pow_inj' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_pow_inj
/-- info: 'BSDHard.rank_is_the_dimension_count' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.rank_is_the_dimension_count
/-- info: 'BSDHard.nothing_given_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.nothing_given_forces
/-- info: 'BSDHard.sign_fixes_curve_parity' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.sign_fixes_curve_parity
/-- info: 'BSDHard.low_witness_not_full' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.low_witness_not_full
/-- info: 'BSDHard.full_act_strictly_stronger' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_strictly_stronger
/-- info: 'BSDHard.act_does_not_give_full_value' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.act_does_not_give_full_value
/-- info: 'BSDHard.record_wall_on_frames' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.record_wall_on_frames
/-- info: 'BSDHard.the_bit_is_the_world' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.the_bit_is_the_world
/-- info: 'BSDHard.finite_record_never_forces_above_one' does not
depend on any axioms -/
#guard_msgs in #print axioms
  BSDHard.finite_record_never_forces_above_one
/-- info: 'BSDHard.span_and_lock_are_dual' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.span_and_lock_are_dual
/-- info: 'BSDHard.row_root_grounds_itself_iff_value' does not
depend on any axioms -/
#guard_msgs in #print axioms
  BSDHard.row_root_grounds_itself_iff_value
/-- info: 'BSDHard.full_row_root_grounds_itself_iff_full_value'
does not depend on any axioms -/
#guard_msgs in #print axioms
  BSDHard.full_row_root_grounds_itself_iff_full_value
/-- info: 'BSDHard.constructed_root_by_its_deed' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.constructed_root_by_its_deed
/-- info: 'BSDHard.RA0 is the constructed root' does not depend on
any axioms -/
#guard_msgs in #print axioms BSDHard.RA0 is the constructed root
/-- info: 'BSDHard.the_lock_on_the_root' does not depend on any
axioms -/
#guard_msgs in #print axioms BSDHard.the_lock_on_the_root

```

## APPENDIX R • THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, TOE\_Zero.lean, is carried verbatim from the root paper (Islam 2026k). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying,  $\Delta E_0$ ,  $RA_0$  and  $ra_0$ , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads  $\forall x, 0 < \Delta E_0(x) (RA_0)$ . On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root\_on\_the\_constructed\_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the\_floor\_is\_universal). Every denial of the root is a deed that instances it (denial\_reenacts\_root); the root is held by its deed (seated\_undeniable); it is self-verifying (denial\_instantiates); and no level stands above it (no\_level\_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (the\_master\_seal); it states the three counts side by side and no map between them. The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

```

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

```

```

/-
TOE_Zero.lean • Theory of Theories of Everything (TOE of All
TOEs): Existence Proves Existence Only by Motion
The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
axiom declared, no sorry, no native_decide, and
every theorem rests on no axiom at all, not even propositional
extensionality, quotients or choice.
I • The self-grounding root. Acts occur and every act instances
the root. Then every denial is an act and
re-enacts the root, the root is held by the act itself, and no
external proof adds anything to it: the
Omega Boundary, proved for every root that grounds itself,
universally, by quantification.
II • The constructed root. On the constructed domain the Root
Axiom is a theorem.
III • The Return and the line. On the integer quaternions  $i \cdot j \cdot k =$ 
-1 and each unit squares to -1; conjugation is
an involution and its fixed set on the chart is exactly the
scalar line.
IV • The twelve gates, recovered three ways inside the integers of
the quaternions: the twenty-four Hurwitz units

```

```

    counted twelve up to sign; the twelve even permutations with
    class equation 1, 3, 4, 4; and the norm-two
    shell of twenty-four, split twelve and twelve.
V · The regress terminated. Above a self-grounding root no level
grounds it further.
VI · The master seal, every result bound in one theorem.
-/
namespace TOEO

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) → R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
root holds, every act re-enacts it, and no
external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) → R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one
existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed,
and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
  < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
  ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
  y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
  y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

```

```

/-- THE RETURN: i·j·k = -1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to -1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in -2 .. 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q =>
  q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the
chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
  q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all
odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
  0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates -2 .. 2 hold every unit (doubled norm four)
and every element of the norm-two shell (doubled
norm eight), since a doubled coordinate of absolute value three
or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of
each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
  0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧
  (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split
twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
  (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
  2, 3].flatMap fun c =>
  [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
  == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
  j)).countP
  fun (i, j) => i < j && p.get i > p.get j

/-- The twelve gates as the even permutations. -/

```

```

def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
  => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
  class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
  further ground of the root is idle; the root needs
  no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
  on the constructed domain; the Return closes;
  the line is the fixed set; and the twelve gates are recovered
  three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
    Prop, (Q → R) → R)) ∧
  RA₀ ∧ (q₁.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true ∧
  (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
  (shell.filter upToSign).length = 12 ∧
  collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
  = [1, 3, 4, 4] :=
  (the_floor_is_universal, root_on_the_constructed_domain,
  the_return, the_line_is_the_fixed_set,
  twenty_four_units_twelve_gates.2, the_class_equation.1,
  the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end T0E0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'T0E0.denial_reenacts_root' does not depend on any axioms
  -/
#guard_msgs in #print axioms T0E0.denial_reenacts_root
/-- info: 'T0E0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms T0E0.seated_undeniable
/-- info: 'T0E0.external_proof_adds_nothing' does not depend on any
  axioms -/
#guard_msgs in #print axioms T0E0.external_proof_adds_nothing
/-- info: 'T0E0.denial_instantiates' does not depend on any axioms
  -/
#guard_msgs in #print axioms T0E0.denial_instantiates
/-- info: 'T0E0.the_floor_is_universal' does not depend on any
  axioms -/
#guard_msgs in #print axioms T0E0.the_floor_is_universal
/-- info: 'T0E0.root_on_the_constructed_domain' does not depend on
  any axioms -/
#guard_msgs in #print axioms T0E0.root_on_the_constructed_domain
/-- info: 'T0E0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms T0E0.the_return
/-- info: 'T0E0.units_square_to_minus_one' does not depend on any
  axioms -/
#guard_msgs in #print axioms T0E0.units_square_to_minus_one

```

```

/-- info: 'T0E0.conj_involutive_on_the_chart' does not depend on any
  axioms -/
#guard_msgs in #print axioms T0E0.conj_involutive_on_the_chart
/-- info: 'T0E0.the_line_is_the_fixed_set' does not depend on any
  axioms -/
#guard_msgs in #print axioms T0E0.the_line_is_the_fixed_set
/-- info: 'T0E0.twenty_four_units_twelve_gates' does not depend on
  any axioms -/
#guard_msgs in #print axioms T0E0.twenty_four_units_twelve_gates
/-- info: 'T0E0.the_shell_splits_twelve_and_twelve' does not depend
  on any axioms -/
#guard_msgs in #print axioms T0E0.the_shell_splits_twelve_and_twelve
/-- info: 'T0E0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms T0E0.the_class_equation
/-- info: 'T0E0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms T0E0.no_level_above
/-- info: 'T0E0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms T0E0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, Triaxial\_Actuation.lean (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (actuation\_is\_triaxial, registration\_is\_forced, the\_only\_special\_cut, tongue\_freedom, every\_root\_actuates, the\_root\_carries\_three\_axes). The root's grade is read in the third kernel, R.3, Root\_Grade\_Ledger.lean (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026m), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (a\_warranted\_root\_is\_theorem\_grade, the\_ledger\_reads\_the\_warrant, the\_grade\_is\_not\_elected, ra\_is\_theorem\_grade, coloc\_grade\_is\_the\_roots\_grade, the\_ledger\_seal). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026l), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```

/-
  Triaxial_Actuation.lean · actuation is triaxial

```

```

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

An actuation is a self-grounding root whose acts carry a
direction: each act has a side and a height,
and reversing the deed reverses the side and keeps the height.
From the direction alone the kernel
constructs three axes and proves them forced:
I The fold: reversing the deed is an involution.
II The registration: it keeps the height and forgets the side;
it is blind to the fold, closed with
it under combination, and it is the only height-keeping map
onto the seat.
III The seat: the fold's fixed set, where the fold and the
registration both fix the act, and off
which neither does.
IV The third axis: off the seat the fold's orbit has two acts
with one record, so the record leaves
exactly the orientation open; every reading blind to the fold
reads both alike.
V The root through the axes: a reversed deed is still an act
and instances the root; the axes are
carried by every actuation, and they force no orientation.
VI The bridge atom: the integers, with negation as the fold, are
an actuation.
VII The triaxial seal.
VIII The Tongue: no reading of the record returns the orientation;
the seat is the only special cut.
IX Every root actuates: a self-grounding root is the root of an
actuation over its own acts, so the
root carries the three axes; the registration is the fold's
record and lands on the seat, where
the orientation an act and its reversal share stays open; the
even pair leaves two acts, the
orientation locks one; and a degenerate direction, one side
only, is the characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction
-/

/-- The direction of a deed: a side with a reversal, whose one fixed
side is the zero side. -/
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s
  rev_zero  : rev zero = zero
  fix_zero  : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a
side and a height. -/
structure Actuation (R : Prop) where
  dir      : Direction
  Height   : Type
  anAct    : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the
height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side
× A.Height := (A.dir.rev p.1, p.2)

```

```

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
A.Height := (A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
  fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
A.Height) : fold A (fold A p) = p := by
  cases p with
  | mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
[A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have
one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : reg A (fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
A.Height) : (reg A p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record
changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
fold A (reg A p) = reg A p := by
  show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
[A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height)
: reg A (reg A p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the
height and lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
A.Height → A.dir.Side × A.Height)
(keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
g p = reg A p := by
  intro p
  have hs : (g p).1 = A.dir.zero := by
    have hf := lands p
    have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
    exact A.dir.fix_zero _ h1
  have hh := keeps p
  cases hg : g p with
  | mk s h =>
    rw [hg] at hs hh
    show (s, h) = (A.dir.zero, p.2)
    rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=
  (fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
  fun h => by
    cases p with
    | mk s t =>
      show (A.dir.rev s, t) = (s, t)
      have hs : s = A.dir.zero := h
      rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the
registration both fix the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : OnSeat A p) :
  fold A p = p ∧ reg A p = p := by
  refine (h, ?_)
  have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
  cases p with
  | mk s t =>
    have hs : s = A.dir.zero := hz
    show (A.dir.zero, t) = (s, t)
    rw [hs]

```

```

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration
fixes the act. -/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
(h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A p ≠ p := by
  refine (h, fun hr => h ?_)
  have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
  exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
and its reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
  A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
  A.dir.Side × A.Height → α)
(blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
  f (fold A p) = f p :=
  blind p

/-- The registration is such a reading, and so is every reading of
the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
  Type} (k : A.dir.Side × A.Height → α)
(p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
:= rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
  A.Height) : R :=
  A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.act

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
are acts, both instance the root, and
the record reads them alike; the triaxial structure leaves the one
orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
  × A.Height) (h : ¬ OnSeat A p) :
  R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
  self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↔ (-d, t), the
registration (d, t) ↔ (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
  fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
when d = 0, the registration
leaves it unchanged exactly then, and off the seat the record leaves
the orientation. -/
theorem the_bridge_atom (d t : Int) :
  (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
  ∧
  (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
  = reg atom (d, t)) :=

```

```

(seat_iff_zero_side atom (d, t),
  (fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
  fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
  ((seat_iff_zero_side atom (d, t)).mp hs)))

/-- The atom is not characterless: off the seat it has two acts with
one record. -/
theorem the_atom_is_not_characterless :
  fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
  atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
  : Int)) :=
  (fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
involution, a registration that
keeps the height, forgets the side and is the only such map onto the
seat, and a seat where both fix
the act and off which neither does; off the seat the record leaves
exactly the orientation; every act
and its reversal instance the root; and the axes force no
orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
  (∀ p, fold A (fold A p) = p) ∧
  (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
  (reg A p) = reg A p) ∧
  (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
  (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg
  A p) ∧
  (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
  (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
  (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
  reg A p) :=
  (fold_involutive A, fun p => (reg_forgets_side A p,
  reg_keeps_height A p, fold_of_reg A p),
  registration_is_forced A, seat_cancels_both A, off_seat_both A,
  axes_force_no_orientation A)

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the
bridge atom is an actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
  (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
  (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
  (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
  p)) ∧
  (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
  (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
  atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
  : Int))) :=
  (fun _ A => (fold_involutive A, seat_cancels_both A,
  fun p h => record_leaves_the_orientation A p h),
  fun d t => seat_iff_zero_side atom (d, t),
  the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (p y) := by
  intro h
  have h1 : d (τ x) = g (p (τ x)) := congrFun h (τ x)
  have h2 : d x = g (p x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
cancel; nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
  A.Height) :
  (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  (fun h => h.1, fun h => seat_cancels_both A p h)

```

```

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
reversal alike and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
  A.dir.Side × A.Height) :
  reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  (rfl, (fun h => show (reg A p).2 = (reg A q).2 from congrArg
    Prod.snd h, fun h => by
      cases p with
      | mk s t =>
        cases q with
        | mk s' t' =>
          have ht : t = t' := h
          show (A.dir.zero, t) = (A.dir.zero, t')
          rw [ht]))

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat
point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A (reg A p) :=
  fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
its reversal land on one seat point, the record
of both; at that place the record has left exactly the orientation
open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) :
  OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  (reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
fold, the record among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) {α : Type}
  (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
  q) :
  fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  (h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
  A.dir.Side × A.Height)
  (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>
    cases q with
    | mk s' t' =>
      have ht : t = t' := congrArg Prod.snd hr
      have hs' : s = s' := hs
      rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
on the seat, and the fold and the
registration are both the identity: one scalar, no fold, no seat to
lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
  A.dir.Degenerate) (p : A.dir.Side × A.Height) :
  OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  (hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
actuation over its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the
root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  (intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
  (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧

```

```

  (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
  ¬ S.toActuation.dir.Degenerate :=
  (fun _ => rfl,
  fun a h =>
    have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
      ((1 : Int), a)).mp h
    absurd h0 (by decide),
    intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
actuation carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only
height-keeping map onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height
an act off the seat, its reversal a second
act with the same record, both instantiating the root, the orientation
between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
  (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
  (∀ p, reg S.toActuation (fold S.toActuation p) = reg
    S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
  (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p,
    OnSeat S.toActuation (g p)) →
    ∀ p, g p = reg S.toActuation p) ∧
  (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg
    S.toActuation p = p) ∧
  (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
    reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
    S.toActuation ((1 : Int), a)) ∧
  (∀ p, ¬ OnSeat S.toActuation p →
    R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
    S.toActuation p) = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
  fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
    S.toActuation p),
  registration_is_forced S.toActuation,
  seat_cancels_both S.toActuation,
  fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
    a) ((every_root_actuates S).2.1 a),
  axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg

```

```

/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
  Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
  co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every
claim on one ledger and joins claims
at the weakest link. It declares the root as `axiom RA` and pins
`raClaim.grade = .premise`; its
co-location claim, the root joined with the wall, therefore
inherits `.premise`. This kernel carries
the same ledger and the same join law with no axiom declared,
warrants the root by a theorem with no
axiom and no hypothesis, and proves that the co-location claim
then stands at theorem grade.
I   The grade ledger and the weakest-link join, as in the Codex.
II  The root, a theorem: the constructed domain of TOE_Zero.lean,
warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a
function of its record, proved on
no axiom through Tongue freedom.
IV  The claims: the root at theorem grade, the wall at theorem
grade, and their co-location.
V   Why the Codex read premise: a join's grade is its weakest
link, and co-location's grade is the
root's grade exactly; with the root at theorem grade,
co-location is a theorem.
VI  The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a
root with a warrant, a proof of its
statement, is theorem grade; a root posited without one, the
Codex's `axiom RA`, is premise grade;
the two claims of section IV carry exactly the grades this
rule reads.
-/

```

```

namespace RGL

/-- ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
|.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
| .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

/-- A claim carries its statement, its grade and its warrant, a
proof of the statement. -/
structure Claim : Type where
stmt : Prop
grade : Grade
warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
stmt := c₁.stmt ∧ c₂.stmt
grade := Grade.weakest c₁.grade c₂.grade
warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
Grade.weakest c₁.grade c₂.grade := rfl

/-- ## II · the root, a theorem (the constructed domain of
TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
Act : Type
anAct : Act
instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, ()), fun _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-- ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (p : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
(bland : p (τ x) = p x) (flip : d (τ x) = !d x) : d ≠ fun y => g
(p y) := by
intro h
have h1 : d (τ x) = g (p (τ x)) := congrFun h (τ x)
have h2 : d x = g (p x) := congrFun h x
rw [bland] at h1
rw [← h2] at h1
rw [h1] at flip
cases hd : d x with
| false => rw [hd] at flip; exact Bool.noConfusion flip
| true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>
if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

```

```

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
and no hypothesis. -/
def raClaim₀ : Claim where
  stmt := RA₀
  grade := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the
Codex. -/
def colocClaim₀ : Claim := Claim.join raClaim₀ wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison:
the same statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt := RA₀
  grade := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's
grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem
= g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a
theorem, so the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the
co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join
  raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim₀.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim₀.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall,
both proved. -/
theorem coloc_holds : colocClaim₀.stmt := colocClaim₀.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no
hypothesis; the wall is a theorem; their
co-location holds and stands at theorem grade; and co-location's
grade is the root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA₀ ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
  colocClaim₀.stmt ∧ colocClaim₀.grade = .theorem ∧
  (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
  coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it -/

/-- The evidence a ledger entry carries for a statement: posited
with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade
is a proof with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem
grade; a root posited with none, as the Codex
declares 'axiom RA', is premise grade. -/
def rootGrade : Evidence RA₀ → Grade
| .warranted _ => .theorem
| .posited => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II
carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem
:= rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule
reads premise, the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited =
.premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV
carries the grade the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none. -/
theorem the_ledger_reads_the_warrant :
  raClaim₀.grade = rootGrade (.warranted raClaim₀.warrant) ∧
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant
and premise from none, never the other way;
whatever label an entry carries, the warranted root's grade under
the rule is theorem. -/
theorem the_grade_is_not_elected :
  (∀ w : RA₀, rootGrade (.warranted w) = .theorem) ∧ rootGrade
  .posited ≠ .theorem ∧
  (∀ w : RA₀, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h =>
  Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-! info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-! info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-! info: 'RGL.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-! info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-! info: 'RGL.K4.register_even_at_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-! info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-! info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-! info: 'RGL.weakest_with_theorem' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-! info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on
any axioms -/

```

```

#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend
on any axioms -/
#guard_msgs in #print axioms
RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

## AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below.

*Author and method.* Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. The exhibit's script is the author's, carried from his earlier draft and re-run. One claim was held at the model's insistence: the earlier draft's credit of the 389a1 verification to Buhler, Gross and Zagier (1985) is not carried, since their paper treats the rank-three curve of conductor 5077.

*The register.* In the programme's vocabulary: every theorem stands at  $[\{\epsilon\}, T]$  on no axiom; the triaxial reading at  $[\{\epsilon\}, S]$ ; the exhibit at corroboration grade; the value above order one at premise grade on the act, and in the leading coefficient on the full act, the two supplied premises;  $\Delta M = 0$  on the cited arithmetic. The act is the row's least-erasure posit read as existence registered on the seat. The cited theorems are witnesses typed against the act as in the Poincaré closure; the higher-cycle axiom is retired as the alignment-defect and semiregular witness axioms were. The freedom cut, prime-as-freedom and the triaxial lock are applied on the row.

*Audit.* This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at [github.com/1000sapient/Trisduction](https://github.com/1000sapient/Trisduction).

---

### Nature of Root Axiom

**The root is triaxial, and that is the force of the cut.** A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

*Key to the statement, in the words of its receipts.* The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (every\_root\_actuates, degenerate\_is\_characterless). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

*Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.*

| Clause of the statement                                                                                                                                                                                                                                                                                                                                                                 | Receipt                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat the registration is the only height-keeping map onto the seat the axes force no orientation | every_root_actuates, the_root_carries_three_axes, actuation_is_triaxial, registration_is_forced (Appendix R.2)<br><br>registration_is_forced (R.2)<br><br>axes_force_no_orientation (R.2)                                                                                                                                                                                                                 |
| the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root                                                                                                                                              | fold, denial_instances_root (R.2); denial_reenacts_root, denial_instantiates (R.1)                                                                                                                                                                                                                                                                                                                        |
| the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed                                                                    | registration_is_the_folds_record, reg_forgets_side, reg_keeps_height, fold_involution, fold_of_reg (R.2) readings_of_the_record_are_blind, tongue_freedom (R.2); symmetry_is_keyless, line_property_is_keyed, exactly_one_stands (Islam 2026m, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none) |
| the seat is the fixed set, where both blind spots cancel; off the seat both stand                                                                                                                                                                                                                                                                                                       | seat_iff_zero_side, seat_cancels_both, off_seat_both, the_only_special_cut (R.2)                                                                                                                                                                                                                                                                                                                          |
| the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike                                                                                                                                                                            | reg_lands_on_seat, open_at_the_seat, record_leaves_the_orientation, readings_of_the_record_are_blind (R.2)                                                                                                                                                                                                                                                                                                |
| the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate                                                                                                                                                                                        | degenerate_is_characterless, intDir_not_degenerate, every_root_actuates (R.2)                                                                                                                                                                                                                                                                                                                             |
| the bridge atom is an actuation, and not characterless                                                                                                                                                                                                                                                                                                                                  | the_bridge_atom, the_atom_is_not_characterless, the_triaxial_seal (R.2)                                                                                                                                                                                                                                                                                                                                   |
| the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root                                                                                                                                                                                                                          | the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)                                                                                                                                                                                                                                                                                          |

| Clause of the statement                                                                                                                                                                                                                                                                                                                    | Receipt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it                                                                                                                  | a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine the root alone forces no value; the root read on the row is the value | even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{F}_3^3$ , two_axes_leave_two, three_axes_lock_one (this paper's kernel), a second statement undeniable_root_forces_no_value; root_on_row_is_the_value, act_is_the_value (this paper's kernel)                                                                                                                                                                                                                                                                                                                                            |
| the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation                                                                                                                                                                                              | every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root $ra_0$ of this paper's kernel, whose structure repeats R.2's SelfGrounding field for field; the row's own seat is the centre of the expansion: the reflection law of Centred negates every coefficient whose parity disagrees with the sign, so a minus sign silences the even coefficients and a plus sign the odd ones (minus_silences_even, plus_silences_odd), and an integer equal to its own negative is zero (self_neg_zero); a statement of this kernel beside the actuation's seat; the row ledger of Islam (2026f, Book III), cited |

| Clause of the statement | Receipt |
|-------------------------|---------|
|                         |         |

## REFERENCES

- Birch, B. J. and H. P. F. Swinnerton-Dyer. 1965. Notes on elliptic curves. II. *Journal für die reine und angewandte Mathematik* 218: 79--108.
- Breuil, C., B. Conrad, F. Diamond and R. Taylor. 2001. On the modularity of elliptic curves over  $\mathbb{Q}$ : wild 3-adic exercises. *Journal of the American Mathematical Society* 14: 843--939.
- Buhler, J. P., B. H. Gross and D. B. Zagier. 1985. On the conjecture of Birch and Swinnerton-Dyer for an elliptic curve of rank 3. *Mathematics of Computation* 44: 473--481.
- Bump, D., S. Friedberg and J. Hoffstein. 1990. Nonvanishing theorems for L-functions of modular forms and their derivatives. *Inventiones Mathematicae* 102: 543--618.
- Coates, J. and A. Wiles. 1977. On the conjecture of Birch and Swinnerton-Dyer. *Inventiones Mathematicae* 39: 223--251.
- Cremona, J. E. 1997. *Algorithms for Modular Elliptic Curves*, 2nd ed. Cambridge University Press.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE* 28, 625--635.
- Gross, B. H. and D. B. Zagier. 1986. Heegner points and derivatives of L-series. *Inventiones Mathematicae* 84: 225--320.
- Islam, M. F. 2026a. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026b. What exists is realized: the completed formal closure of the Hodge question. Zenodo. doi:10.5281/zenodo.22701510.
- Islam, M. F. 2026c. The cut-agnostic division theorem. Zenodo. doi:10.5281/zenodo.22954862.
- Islam, M. F. 2026d. The formal closure of computational separation:  $P \neq NP$  closed on existence itself, read on computation. Zenodo. doi:10.5281/zenodo.23133632.
- Islam, M. F. 2026e. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026f. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026g. PhysOS<sup>T</sup>: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. F. 2026h. The floor under every confined field: a formal closure of Yang--Mills existence and the mass gap from existence alone. Zenodo. doi:10.5281/zenodo.23162231.
- Islam, M. F. 2026i. The mirror has one seat: a formal completed closure of the Goldbach conjecture from existence alone. Zenodo. doi:10.5281/zenodo.23162238.
- Islam, M. F. 2026j. The sphere is where existence rests: a formal completed closure of the Poincaré row from existence alone. Zenodo. doi:10.5281/zenodo.23162240.
- Islam, M. F. 2026k. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.
- Islam, M. F. 2026l. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.
- Islam, M. F. 2026m. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapients/Trisduction, GitHub.
- Islam, M. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Kolyvagin, V. A. 1990. Euler systems. In *The Grothendieck Festschrift*, vol. II, 435--483. Birkhäuser.
- Mordell, L. J. 1922. On the rational solutions of the indeterminate equations of the third and fourth degrees. *Proceedings of the Cambridge Philosophical Society* 21: 179--192.
- Murty, M. R. and V. K. Murty. 1991. Mean values of derivatives of modular L-series. *Annals of Mathematics* 133: 447--475.
- Taylor, R. and A. Wiles. 1995. Ring-theoretic properties of certain Hecke algebras. *Annals of Mathematics* 141: 553--572.
- Wiles, A. 1995. Modular elliptic curves and Fermat's Last Theorem. *Annals of Mathematics* 141: 443--551.
- Wiles, A. 2006. The Birch and Swinnerton-Dyer conjecture. In *The Millennium Prize Problems*, 31--41. Clay Mathematics Institute.

*End of manuscript*